# VALOR

Vital and Atmospheric Logic for Offline Rescue

## The Engineer's Guide

How we built an offline, FPGA-assisted disaster wearable
and how to understand, explain and defend every part of it

| | |
|---|---|
| **Project** | SIH26181 — Smart India Hackathon 2026 |
| **System class** | Heterogeneous edge SoC: FPGA Verilog RTL + ESP32-S3 FreeRTOS C11 + zero-cloud INT8 TinyML |
| **Audience** | B.Tech ECE students, beginner to confident, including the jury defense |
| **Format** | Study guide + defense manual with diagrams, worked examples and Q&A |

*Diagrams are redrawn for teaching; waveforms marked “illustrative” are synthetic, not measured data.*

# Contents

0 How to Read This Guide . . . 3
1 The Problem and Why It Matters . . . 5
2 What VALOR Does . . . 7
3 System Architecture . . . 9
4 The Two Hardware Targets . . . 12
5 The Link Between the ESP32 and the FPGA . . . 14
6 Where Else the Protocols Are Used . . . 18
7 FPGA and RTL Fundamentals Used in VALOR . . . 19
8 The 8-Tap Moving Average Filter . . . 22
9 The Peak Detector Finite State Machine . . . 25
10 The Other FSMs in the Design . . . 29
11 Timing, Resources and Synthesis Results . . . 32
12 The ESP32-S3 Firmware . . . 35
13 The AI Layer (TinyML) . . . 38
14 Clinical and Disaster Logic (the Rule-Based Parts) . . . 42
15 Sensors in Simple Terms . . . 45
16 Location and Connectivity . . . 48
17 Verification and Testing . . . 49
18 Demo Walkthrough . . . 50
19 Judge Q&A; Playbook — 30 Likely Questions . . . 52
20 Honest Limitations . . . 56
21 Glossary . . . 57
22 Who Should Be Able to Explain What . . . 59
23 Open Questions and Reconciliations . . . 60
Appendix A · Quick-Reference Card . . . 61
Appendix B · Self-Test . . . 62
Appendix C · Reconciliation Notes from the Editing Pass . . . 62

START HERE

# 0 How to Read This Guide

I am one of the engineers who built VALOR, and I wrote this guide for the students who will carry it forward. You do not need to memorise pin numbers, memory addresses or register bits. You do need to understand **why** the hardware is shaped the way it is, **how** signals move through it, **what** each state machine does, and **where** the engineering trade-offs are. If you can explain those four things, you can defend the project in front of any technical panel.

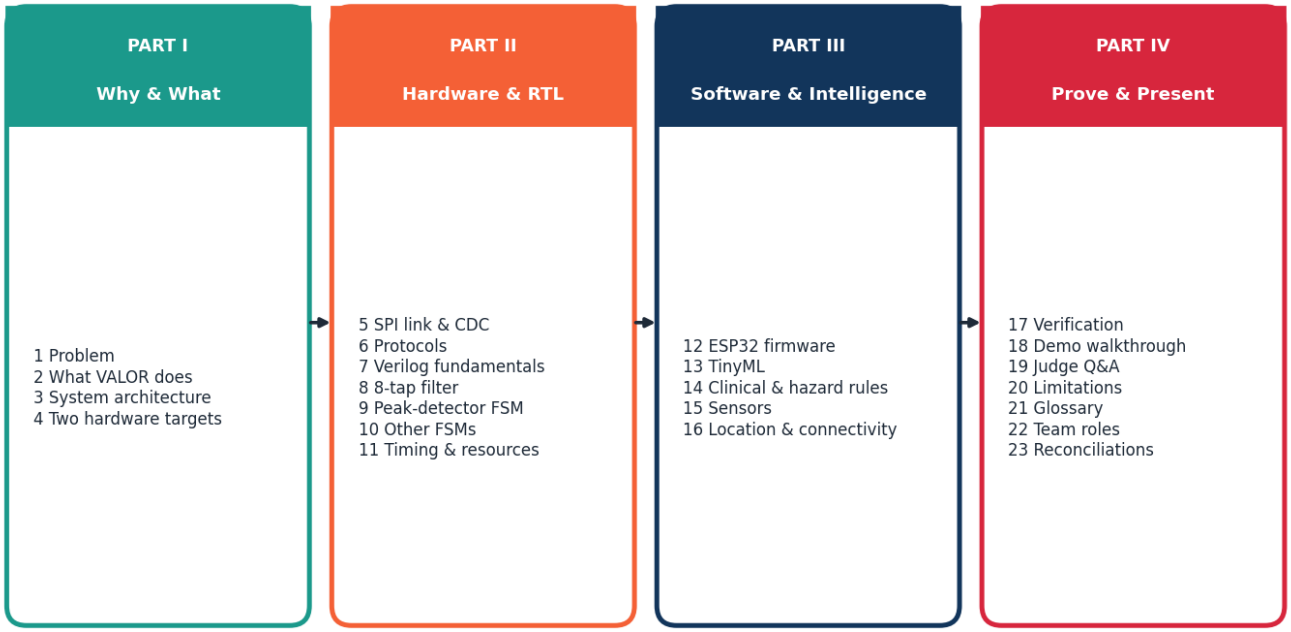



*Figure 0.1 — The guide in four parts. Chapter numbers match the original VALOR study text and the Judge Q&A playbook.*

## Three reading paths

| Your situation | Read these chapters | Why |
|---|---|---|
| **First read** (about 1 hour) | 0 – 4, then 5, 8, 9, 12, 13, 14 | The big picture plus the core chain: link → filter → peak detector → firmware → AI. |
| **Deep technical review** (before the Verilog / RTL jury) | 5, 7, 8, 9, 10, 11 | FSM diagrams, clock-domain crossing, synthesis and timing reports. |
| **Pre-demo rehearsal** (30 minutes before judging) | 18, 19, 20, 22 | Demo script, the 30-question playbook, honest limitations, who owns what. |

## The five-minute summary

- **The problem.** In cyclones, floods and chemical fires the cell towers fail, the grid dies, and heat, smoke and cold accelerate physical collapse. Commercial smartwatches depend on the cloud, and their operating systems add 5–10 ms of timing jitter that destroys heart-rate-variability metrics.
- **The solution.** VALOR is a wearable that works 100 % offline. It fuses optical vitals (pulse, $SpO_2$) with hazards (smoke, heat, humidity, pressure drops, falls).
- **The hardware.** A “dual brain”. A sub-dollar Renesas ForgeFPGA runs Verilog at 50 MHz and detects each heartbeat with 20 ns accuracy using 0 DSP slices and 0 Block RAM. A dual-core ESP32-S3 (240 MHz, FreeRTOS, C11) does fusion, a 619-byte INT8 neural network, clinical rules and the SOS logic.
- **The output.** An OLED shows live vitals and an emergency card; a blue LED flashes on every heartbeat, driven directly by the FPGA; and the ESP32 hosts an open Wi-Fi hotspot that serves a zero-install dashboard to any rescuer's phone.

**HONEST DISCLOSURE**

**The reporting rule for every number in this guide.** State the MIMIC-III result as **58.18 % sensitivity / 98.19 % specificity**. The 94.11 % accuracy figure hides that pair, because only 10.2 % of the labels are positive. And **never present a skin-temperature estimate as a core temperature.**

**ENGINEER'S NOTE**

**Companion documents.** The source study text points to four shorter, more frequently updated files: *DEMO_SHEET.md* (newest, trust it when files disagree), *JUDGE_QA.md*, *DEMO_RUNBOOK.md* and *SHRIKEFI_HARDWARE_CONNECTIONS.md* (authoritative pin map). They were not supplied with this guide. Where I found numbers in the long text that disagree with each other, I list them in Appendix C so you can check them against those files and the code.

**ENGINEER'S NOTE**

**How the chapters are built.** Every chapter opens with what you will be able to do, explains the idea in plain words first, then shows the real design, and closes with an *In one line* summary and the questions a judge is most likely to ask.

PART I · WHY AND WHAT

# 1 The Problem and Why It Matters

**AFTER THIS CHAPTER YOU CAN…**

- ✓ explain why cloud-dependent wearables fail in a disaster
- ✓ name the four environmental hazards VALOR watches and how each one hurts the body
- ✓ define alarm fatigue and say how sensor fusion avoids it

## 1.1 Disasters destroy communications first

When a tropical cyclone, flash flood, industrial fire or earthquake hits, backhaul fibre tears, cellular base stations flood or lose grid power, and cloud servers become unreachable. A wearable that needs a cloud API to analyse data or raise an alarm becomes a dead brick at exactly the moment it is needed.

## 1.2 The environment is part of the medical problem

| Hazard | What happens | What VALOR watches |
|---|---|---|
| **Heat and humidity** | Above about 40 °C with high humidity, sweat cannot evaporate; core temperature climbs → heat exhaustion, heat stroke. | BME280 temperature + humidity → Heat Index; heart rate; RMSSD |
| **Toxic smoke (PM2.5)** | Particles ≤ 2.5 µm cross into the blood → hypoxaemia and autonomic depression. | PMS5003 PM2.5; $SpO_2$; heart rate |
| **Falling pressure** | A rapid drop signals an approaching cyclone or squall line. | BME280 pressure slope over 180 min |
| **Cold-water immersion** | Floodwater → cold shock, hypothermia, sudden bradycardia. | Cold-stress exposure proxy (capped at HIGH) |

## 1.3 The alarm-fatigue trap of single-sensor devices

A fitness tracker looks only at heart rate. 135 BPM while jogging in a cool park is healthy; the *same* 135 BPM at 44 °C with high humidity is impending heat stroke. Conversely a wall-mounted air sensor knows the city's pollution but nothing about the person breathing it. VALOR couples the two so it can tell safe exertion from multi-hazard collapse.

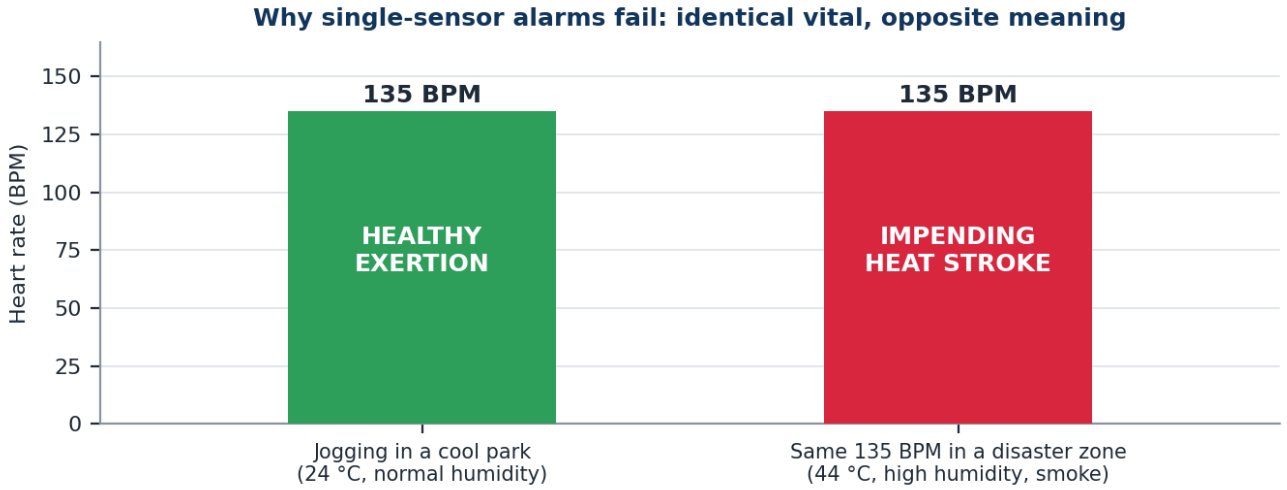



*Figure 1.1 — Context turns the same number into opposite decisions. (Conceptual illustration.)*

**IN ONE LINE**

Disasters destroy networks while heat, smoke and storms accelerate collapse; VALOR couples vitals with atmospheric hazards, 100 % offline, at the silicon edge.

**JUDGE MIGHT ASK**

**Q.** *Why not just use an Apple Watch or Garmin for first responders?*

**A.** They rely on cloud analytics, have no PM2.5 sensor, do not compute barometric cyclone trends, run closed operating systems with 5–10 ms scheduling jitter, and cannot host an open local Wi-Fi triage page for rescue teams. Frame it as context: they are built for consumer fitness, VALOR for disaster rescue.

**Q.** *What is alarm fatigue and how does VALOR prevent it?*

**A.** It is what happens when simple threshold alarms fire constantly during harmless exertion until people ignore them. VALOR evaluates vitals against ambient heat and humidity, so alarms fire only on real physiological strain.

PART I · WHY AND WHAT

# 2 What VALOR Does

**AFTER THIS CHAPTER YOU CAN…**

- ✓ describe the Measure → Decide → Act loop
- ✓ list every quantity VALOR measures and every one it derives
- ✓ explain what "100 % offline" really means in engineering terms

VALOR measures physiological and atmospheric variables on the body, evaluates them with on-chip logic and machine learning, and raises local alarms with no external dependency.

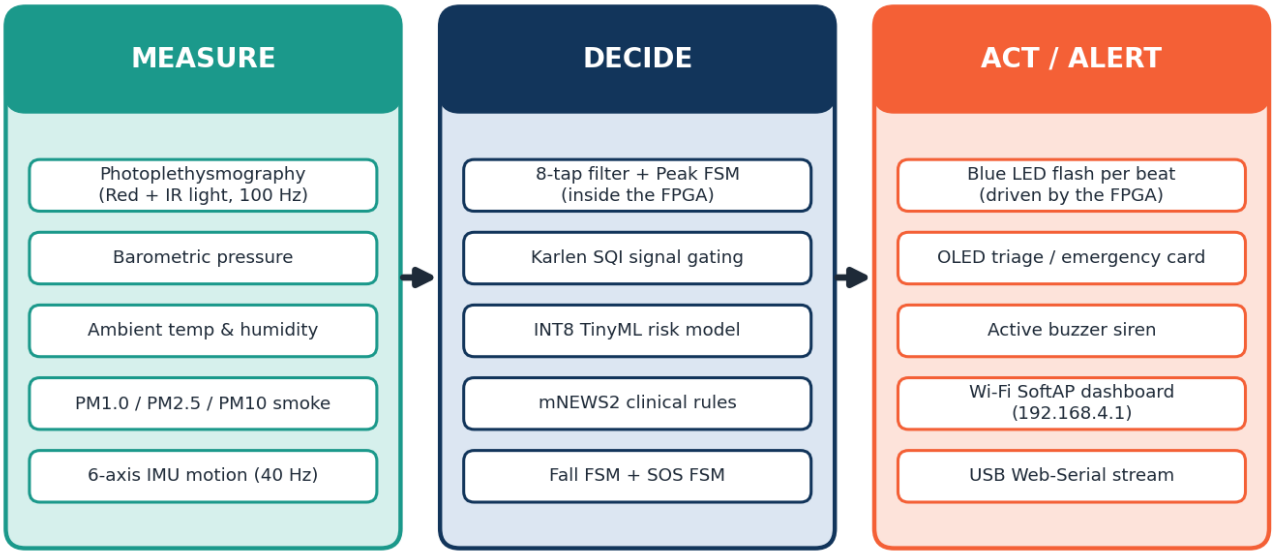



*Figure 2.1 — The whole system as a three-stage loop.*

## 2.1 What it measures

| Signal | How | Rate |
|---|---|---|
| Arterial pulsatile blood volume (PPG) | Infrared and red light absorption through the fingertip capillary bed | 100 Hz |
| Barometric pressure | 0.01 hPa resolution; tracks fall rate over hours | 1 Hz |
| Ambient temperature and humidity | Environmental thermal stress | 1 Hz |
| Particulate matter PM1.0 / 2.5 / 10 | 90° laser scattering of suspended smoke and dust | 1 Hz |
| Motion | 3-axis acceleration + 3-axis angular rate: gait, stillness, falls | 40 Hz |

## 2.2 What it decides on the device

- **Inter-beat interval (IBI)** — the time between consecutive systolic peaks, in units of 20 ns.
- **Heart rate and HRV** — BPM, and RMSSD in ms (a marker of autonomic nervous system health).
- **$SpO_2$** — from the ratio-of-ratios of red and infrared light curves.
- **Respiration rate** — extracted from respiratory sinus arrhythmia (breathing modulates the beat-to-beat interval).
- **Signal Quality Index (SQI)** — checks pulse shape and discards motion-corrupted data *before* any clinical algorithm runs.
- **Multi-hazard triage** — heat strain, toxic pollution, flash-flood / cold stress, impending cyclone.
- **Clinical urgency** — a modified National Early Warning Score 2 (mNEWS2).

## 2.3 What alerts it raises

- **Hardware beat flash** — a blue LED pulses 50 ms on every confirmed crest, straight from the FPGA.
- **OLED emergency card** — condition, vitals, elapsed time and location.
- **Audible siren** — an active buzzer for falls and critical alerts.
- **Rescuer dashboard** — live vitals, 24-hour hourly trends and alerts over the local Wi-Fi network `VALOR-0081`.

## 2.4 Why “offline” is a real claim

There are no external API calls, DNS lookups, cloud databases or CDN dependencies. All HTML, CSS, JavaScript and neural-network weights are compiled into the ESP32-S3's flash. If the global internet vanished, VALOR would behave exactly the same.

**IN ONE LINE**

VALOR captures vitals and hazards, filters them in FPGA hardware, classifies multi-hazard risk on a dual-core MCU, and shows triage alerts locally.

**JUDGE MIGHT ASK**

**Q.** *Does the web dashboard need internet or an app?*

**A.** No. The ESP32-S3 broadcasts its own access point (VALOR-0081) and serves the page from flash. A phone joins the Wi-Fi and opens 192.168.4.1.

**Q.** *Why does the device withhold vitals when the signal is noisy?*

**A.** The Karlen Signal Quality Index screens each pulse. If motion corrupts the optical path (SQI < 0.70) the device refuses to compute HRV rather than publish garbage that could cause false alarms.

PART I · WHY AND WHAT

# 3 System Architecture

**AFTER THIS CHAPTER YOU CAN…**

- ✓ draw the block diagram of VALOR from memory
- ✓ trace one heartbeat from photon to alarm in thirteen steps
- ✓ justify splitting the work between an FPGA and a microcontroller

VALOR divides labour between dedicated digital logic (the FPGA) and general-purpose software (the MCU). Anything that needs *exact, repeatable timing* goes to hardware; anything that needs *floating-point maths, networking or flexibility* goes to software.

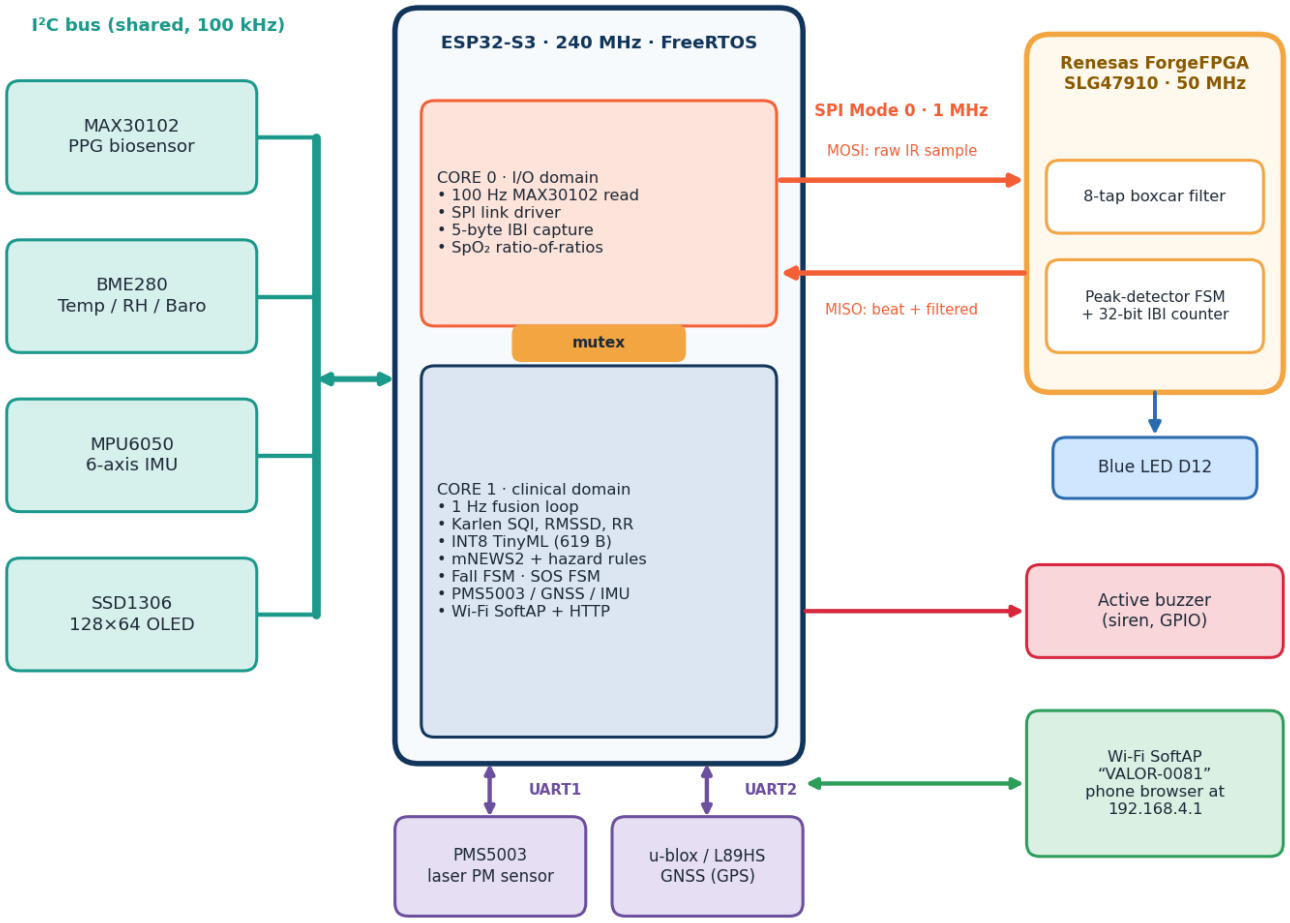



*Figure 3.1 — Project VALOR hardware. Teal = shared I²C bus, orange = SPI link, purple = UART streams.*

## 3.1 One heartbeat, thirteen steps




*Figure 3.2 — The data path from sensor sample to alert. Colours show which part of the system does each step.*

| # | Step | What really happens |
|---|---|---|
| 1 | Optical capture | Every 10 ms the MAX30102 captures an 18-bit reading of reflected infrared light from the fingertip. |
| 2 | I²C ingestion (Core 0) | The ESP32 reads the sample, normalises it to an 8-bit range and prepares an SPI transaction. |
| 3 | SPI to the FPGA | Core 0 drives chip-select low and clocks the byte out on MOSI at 1 MHz. |
| 4 | Hardware filter | `moving_average_8tap.v` smooths noise with a running-sum pipeline. |
| 5 | Peak detection | `ppg_peak_detector.v`: a 4-state FSM waits for the signal to rise above an armed threshold and then fall by at least 25 % of the running peak. |
| 6 | 20 ns timestamp | On a confirmed peak the FPGA latches a 32-bit counter running at 50 MHz, asserts the beat flag and stretches a 50 ms pulse onto the blue LED (D12). |
| 7 | Full-duplex return | MISO returns {beat, filt[6:0]}; after a beat, a 5-byte burst follows: magic 0xC3 + the 32-bit cycle count. |
| 8 | Mutex hand-off | Core 0 converts cycles to an IBI in ms, checks it is physiological (30–250 BPM), appends it to a ring buffer and releases `s_data_mutex`. |
| 9 | Hazard polling (Core 1) | BME280 (I²C), PMS5003 (UART1), MPU6050 (I²C) and the GNSS receiver (UART2) are read concurrently. |
| 10 | Signal conditioning | Karlen SQI; if SQI ≥ 0.70 compute RMSSD and respiration rate. PM2.5 gets an EPA/AirGradient correction; pressure gets a 180-minute least-squares slope. |
| 11 | INT8 inference | Features [HR, RMSSD, $SpO_2$, temp, humidity, PM2.5] → 619-byte MLP → heat, pollution and flood/cold scores. |
| 12 | Rules override | mNEWS2 and hard limits ($SpO_2$ < 88 %, pressure fall ≤ −2.0 hPa/h) force CRITICAL regardless of the network. |
| 13 | Local action | A sustained CRITICAL state or a confirmed fall moves the SOS FSM from ARMED to ACTIVE: siren, OLED emergency card, and an updated /api/status for any connected phone. |

**ANALOGY**

Think of an orchestra. The FPGA is the metronome and the percussion — it must never be late by even a hair. The ESP32 is the conductor and the strings — it interprets, adapts and talks to the audience. A conductor can keep time, but only roughly; a metronome cannot interpret a score.

**IN ONE LINE**

Optical signals flow sensor → MCU → FPGA filters and peak detector → back to the FreeRTOS cores for fusion and INT8 inference → out to OLED, buzzer and an offline browser.

**JUDGE MIGHT ASK**

**Q.** *Why split the design across two chips instead of doing everything on the ESP32-S3?*

**A.** FreeRTOS scheduling creates unpredictable jitter (up to 5 ms per tick). HRV depends on timing differences between beats. The FPGA gives deterministic 20 ns hardware timing independent of CPU load.

**Q.** *How does Core 0 pass 100 Hz PPG data to Core 1 without crashing?*

**A.** Both cores share an in-memory structure protected by a FreeRTOS mutex (`s_data_mutex`). Core 0 writes and releases quickly; Core 1 takes short snapshots at 1 Hz.

PART I · WHY AND WHAT

# 4 The Two Hardware Targets

**AFTER THIS CHAPTER YOU CAN…**

- ✓ explain why the same RTL lives on two different chips
- ✓ state which target is the golden reference and which is the demo
- ✓ list the architectural differences between them

VALOR exists in two implementations: a verified AMD Xilinx Zynq-7000 baseline and a deployable Renesas ForgeFPGA port on the Vicharak ShrikeFi board. Good hardware engineering separates **proving the algorithm** from **optimising the cost**, and these two targets do exactly that.

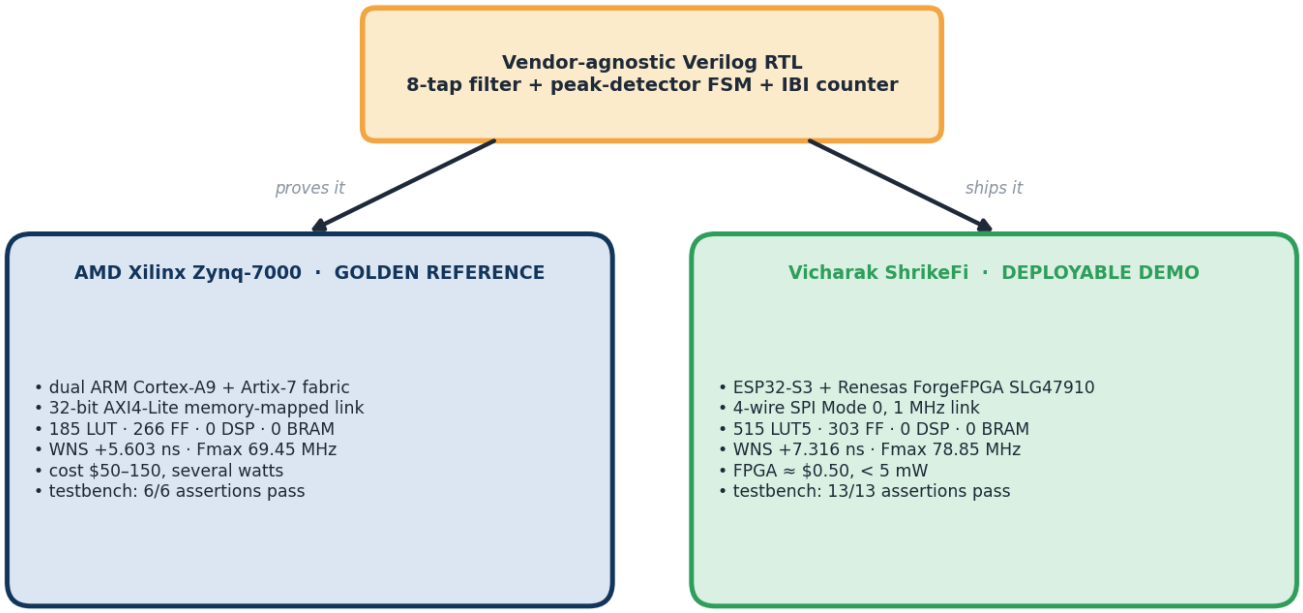



*Figure 4.1 — One vendor-agnostic design, two targets: one to prove it, one to ship it.*

## 4.1 Zynq-7000: the golden reference

- Part `xc7z020clg400-1`: dual ARM Cortex-A9 plus Artix-7 fabric, joined by a 32-bit memory-mapped AXI4-Lite bus.
- Footprint: 185 slice LUTs, 16 LUTRAMs, 266 flip-flops, **0 DSP48, 0 BRAM**.
- Timing (Vivado ML 2022.2, out-of-context): WNS **+5.603 ns**, WHS +0.184 ns, Fmax **69.45 MHz** against a 50 MHz target. A full block-design re-run in Vivado 2026.1 closed post-route at WNS +12.836 ns (1,806 endpoints).
- Verification: `tb_ppg_system.v`, 6/6 self-checking assertions.

## 4.2 ShrikeFi: the deployable demo

- ESP32-S3 (dual-core Xtensa LX7, 240 MHz, single-precision FPU) plus Renesas ForgeFPGA SLG47910 with **1,120 LUT5s** in 140 CLBs.
- Current fit: **515/1120 LUT5s (45.98 %)**, 303 flip-flops (16.33 %), 105/140 CLBs (75.00 %).
- Timing: WNS **+7.316 ns**, TNS 0 ps, Fmax **78.85 MHz** against the 50 MHz SDC constraint.
- Cost and power: the FPGA is about $0.50 and draws under 5 mW — viable for a battery wearable. **This board is the demo shown to judges.**

## 4.3 Differences at a glance

| Feature | Zynq-7000 baseline | ForgeFPGA ShrikeFi port |
|---|---|---|
| **Primary role** | Golden reference, formal RTL verification | Physical demonstration, deployable product |
| **Host processor** | Dual ARM Cortex-A9 (hard processing system) | ESP32-S3 dual-core Xtensa LX7 |
| **Interconnect** | 32-bit memory-mapped AXI4-Lite (0x43C00000) | 4-wire 8-bit SPI, Mode 0, 1 MHz |
| **Sample in** | Write to register REG_IR_RAW | 8-bit MOSI byte |
| **Filtered out** | Read register REG_IR_FILT | MISO bits [6:0] |
| **Beat flag** | REG_STATUS_THRESH[0] | MISO bit 7 |
| **IBI transfer** | Read 32-bit REG_IBI_CYCLES | 5-byte burst (0xC3 + 32-bit big-endian cycles) |
| **Threshold** | Written at run time to REG_STATUS_THRESH[15:8] | Hard-wired at synthesis (8'd120) |
| **Interrupts** | Hardware interrupt line to the ARM GIC | Polled SPI status bit (no interrupt line) |
| **DSP / BRAM** | 0 / 0 | 0 / 0 |

**IN ONE LINE**

The Zynq-7000 is the verified golden reference proving timing closure and zero DSP use; the ShrikeFi is the low-cost, battery-friendly physical demo.

**JUDGE MIGHT ASK**

**Q.** *Why synthesise on Zynq if the demo board uses a ForgeFPGA?*

**A.** Vivado gave industrial-grade timing reports (WNS +5.603 ns) and proved the RTL is vendor-agnostic with 0 DSP and 0 BRAM. We then ported the identical RTL to the sub-dollar ForgeFPGA for a wearable form factor.

**Q.** *Can the peak threshold be changed at run time on ShrikeFi?*

**A.** No. On Zynq it is programmable over AXI4-Lite; on ShrikeFi it is hard-wired to 8'd120 to save gates in the 1,120-LUT fabric. The driver call `shrikefi_set_threshold()` stores the value in memory but does not write hardware.

PART II · HARDWARE AND RTL

# 5 The Link Between the ESP32 and the FPGA

**AFTER THIS CHAPTER YOU CAN…**

- ✓ name the four SPI wires and what each carries
- ✓ explain why SPI beat I²C and UART for this link
- ✓ read an SPI Mode 0 timing diagram
- ✓ decode the MOSI byte, the MISO byte and the 5-byte IBI burst
- ✓ explain metastability and how a 2-flip-flop synchroniser fixes it

The ESP32-S3 ↔ ForgeFPGA link is the central nervous system of VALOR. It runs over **internal PCB traces** on the ShrikeFi board, so no jumper wires are involved.

## 5.1 The wires

| Signal | Driven by | Job |
|---|---|---|
| `spi_sck` | ESP32 | Shift clock. 1 MHz in normal operation; 16 MHz only while loading the bitstream at boot. |
| `spi_ss_n` | ESP32 | Active-low slave select; frames each byte or burst transaction. |
| `spi_mosi` | ESP32 | Raw 8-bit optical PPG sample, MCU → FPGA. |
| `spi_miso` | FPGA | Beat flag + filtered sample, and the IBI burst bytes, FPGA → MCU. |
| FPGA_EN / FPGA_PWR | ESP32 | Power gating and configuration enable, used only while programming the FPGA at boot. |

- **No reset wire.** The FPGA makes its own reset with a 255-cycle power-on-reset counter on its internal 50 MHz oscillator.
- **No interrupt wire.** A heartbeat is signalled *inline* on bit 7 of the MISO byte.

## 5.2 Why SPI?

| Feature | SPI (chosen) | I²C (rejected) | UART (rejected) |
|---|---|---|---|
| **Wires** | 4 (SCK, SS_n, MOSI, MISO) | 2 (SDA, SCL) | 2 (TX, RX) |
| **Speed** | 1–16 MHz in this design | 100–400 kHz | 9,600–115,200 baud |
| **Duplex** | **Full duplex** | Half duplex | Full duplex |
| **Addressing** | None — dedicated select line | 7/10-bit address before every transfer | None |
| **Clocking** | Synchronous, master supplies clock | Synchronous, master supplies clock | **Asynchronous** — both sides need matched baud clocks |
| **FPGA logic cost** | **Tiny** — an 8-bit shift register | **High** — address match, ACK, open-drain | Moderate — baud generator + oversampler |
| **Pull-ups** | None (push-pull) | Mandatory (slows edges) | None |

- **Why not I²C?** It needs an address byte and an ACK around every data byte, it is half-duplex (the MCU cannot send a new sample while reading the old result), and an I²C slave costs LUTs the 1,120-LUT FPGA cannot spare.
- **Why not UART?** No shared clock: both chips need independent clocks matched within about 2 %. If the FPGA's on-chip RC oscillator drifts with temperature, framing breaks.
- **Why SPI fits.** It is a pure shift register: while the master shifts a bit out on MOSI, the target shifts one in on MISO. The master owns the clock, so there is no baud drift.

## 5.3 SPI Mode 0

Mode 0 means **CPOL = 0** (clock idles LOW) and **CPHA = 0** (data is *sampled* on the first, rising edge and *changes* on the falling edge). MISO is pre-loaded with bit 7 before the first rising edge.

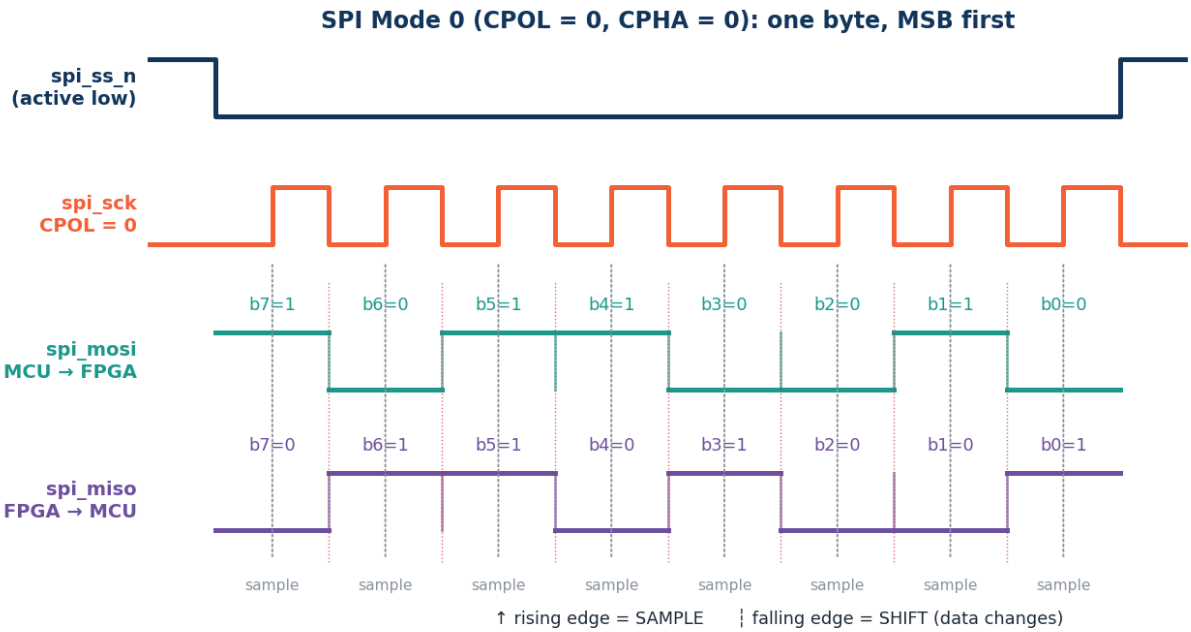



*Figure 5.1 — One SPI Mode 0 byte, MSB first. Both sides sample on rising edges and shift on falling edges.*

## 5.4 The 8-bit full-duplex frame

**Every 10 ms (100 Hz) the MCU sends exactly one byte and gets exactly one byte back**

**MOSI byte (ESP32 → FPGA)**

| sample[7:0] · raw 8-bit infrared PPG sample |
|---|

**MISO byte (FPGA → ESP32), returned during the same 8 clocks**

| 7<br>BEAT | bits 6…0 · filt_sample[6:0] (lower 7 bits of filtered PPG → wraps at 128) |
|---|---|

*Bit 7 = beat_latched: HIGH for exactly one transaction when the FSM confirmed a systolic crest.*

**If BEAT = 1 → five extra transactions (MOSI dummy 0x00) return the cycle-accurate IBI:**

| 0xC3<br>magic | IBI[31:24] | IBI[23:16] | IBI[15:8] | IBI[7:0] |
|---|---|---|---|---|
| burst byte 0 | burst byte 1 | burst byte 2 | burst byte 3 | burst byte 4 |

1 FPGA tick at 50 MHz = 20 ns → IBI timestamp resolution = 20 ns. A wrong first byte (≠ 0xC3) means “not a real burst”:

the driver rejects it and falls back to the CPU microsecond timer, so a noise glitch cannot corrupt HRV.

*Figure 5.2 — One byte out, one byte back, every 10 ms; plus the five-byte burst that follows a beat.*

- **MOSI** carries the raw 8-bit infrared sample.

- **MISO bit 7** (`beat_latched`) is HIGH for exactly one transaction when the peak FSM confirmed a crest. **Bits [6:0]** carry the lower 7 bits of the filtered sample.
- **Why the diagnostic waveform wraps:** only 7 bits remain for data, so the returned waveform wraps at 128 (a sawtooth on a raw serial plot). The firmware treats it as diagnostic only; the FPGA's own peak detector uses the full unclipped 8-bit signal.
- **Single-channel link:** only the infrared channel crosses to the FPGA. The red channel is conditioned in software on the ESP32 for $SpO_2$ (`shrikefi_write_red_sample()` only caches it; `shrikefi_write_ir_sample()` triggers the SPI transfer).
- **The IBI burst:** after a beat the driver clocks five more bytes (MOSI dummy 0x00). The FPGA returns 0xC3, then the 32-bit cycle count big-endian. At 50 MHz one cycle is **20 ns**. If byte 0 is not 0xC3 (older bitstream, or a glitch) the driver falls back to a CPU microsecond timer.

**COMMON TRAP**

**The retired "nibble protocol".** Early notes describe a 4-bit parallel bus with strobe, direction pins and a 0x1–0x8 command codebook. It was retired *before it was built*: it would have used 6–8 of the FPGA's 19 GPIOs, needed bit-banging, and the ESP32-S3 already has hardware SPI (SPI2_HOST). If an old diagram shows a nibble bus, it is history. The real link is pure 8-bit SPI.

## 5.5 Two clock domains and metastability

The ESP32 generates `spi_sck` at 1 MHz. The FPGA fabric runs on its own internal 50 MHz oscillator. These clocks are unrelated, so every incoming SPI pin is *asynchronous* to the FPGA.

**ANALOGY**

Photographing a spinning coin. If you snap it while it is flat you see heads or tails. If you snap it at the instant it stands on its edge, the picture is a blur. A flip-flop that samples a wire while that wire is changing can land in the same "on edge" state — neither 0 nor 1 — for a while. That is **metastability**.

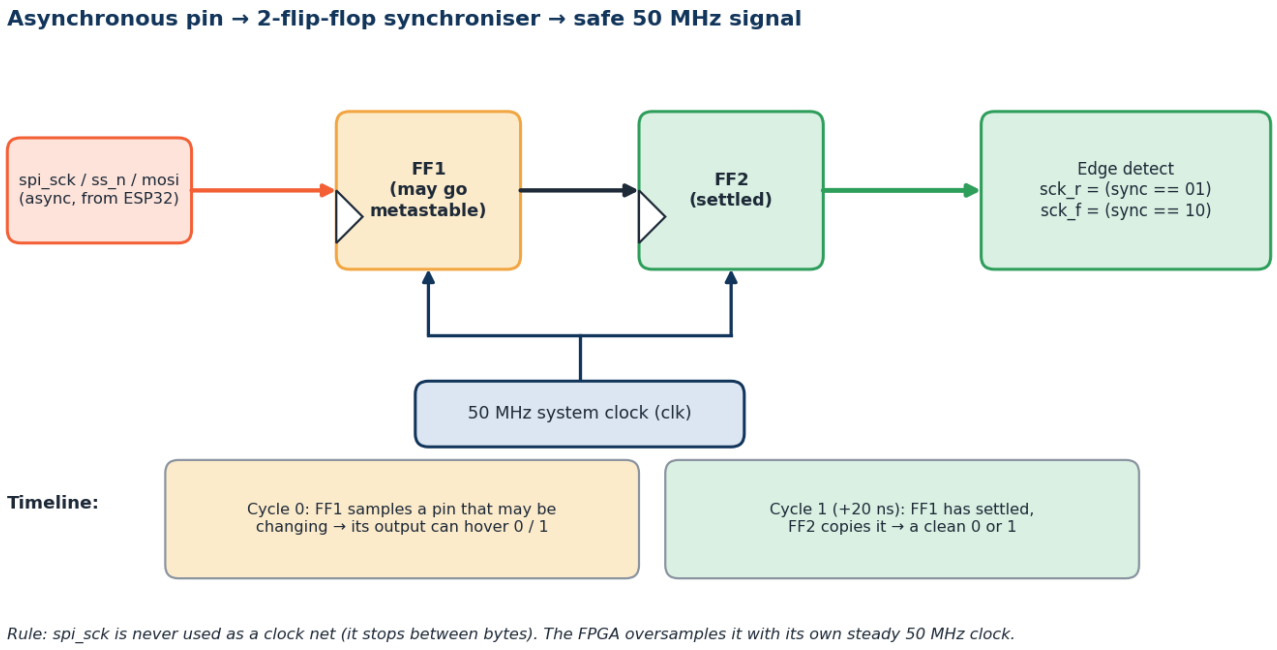



*Figure 5.3 — Two flip-flops in series give the first one a full 20 ns to settle before the second one copies it.*

```
reg [1:0] sck_sync;  reg [1:0] ss_sync;  reg [1:0] mosi_sync;
always @(posedge i_clk or negedge i_rst_n) begin
    if (!i_rst_n) begin sck_sync <= 2'b00; ss_sync <= 2'b11; mosi_sync <= 2'b00; end
    else begin
        sck_sync  <= {sck_sync[0],  i_sck};
        ss_sync   <= {ss_sync[0],   i_ss_n};
        mosi_sync <= {mosi_sync[0], i_mosi};
    end
end
wire sck_r = (sck_sync == 2'b01);   // rising edge  (was 0, now 1)
wire sck_f = (sck_sync == 2'b10);   // falling edge (was 1, now 0)
```

*Listing 5.1 — hardware/shrikefi/spi_target.v: synchronisers and edge detectors. All shifting happens on the single-cycle pulses sck_r and sck_f.*

The FPGA never uses `spi_sck` as a clock net — it stops and starts, which violates FPGA clocking rules. It **oversamples** the signal with its steady 50 MHz clock instead.

**IN ONE LINE**

A 4-wire, full-duplex SPI Mode 0 link at 1 MHz, with asynchronous inputs made safe by 2-stage synchronisers on the FPGA's 50 MHz clock.

**JUDGE MIGHT ASK**

**Q.** *What if the ESP32 ran SPI at 30 MHz?*

**A.** It would fail. The FPGA oversamples SCK with a 50 MHz clock; the Nyquist limit is 25 MHz and, with a 2-flip-flop synchroniser, practical operation needs SCK well below that (about 5–10 MHz or less).

**Q.** *What if noise flips a bit on MISO?*

**A.** If the beat flag is wrongly 1, the ESP32 starts the 5-byte burst. The burst must begin with 0xC3; a corrupted burst fails that check and the firmware discards it and uses its own timer.

**Q.** *Why not a 16- or 32-bit parallel bus?*

**A.** Wide traces, skew risk and almost all of the 19 GPIOs. SPI has plenty of bandwidth: 8 µs of transfer every 10 ms is a 0.08 % bus duty cycle.

PART II · HARDWARE AND RTL

# 6 Where Else the Protocols Are Used

**AFTER THIS CHAPTER YOU CAN…**

- ✓ match each sensor to its bus
- ✓ explain why a bus is shared or dedicated

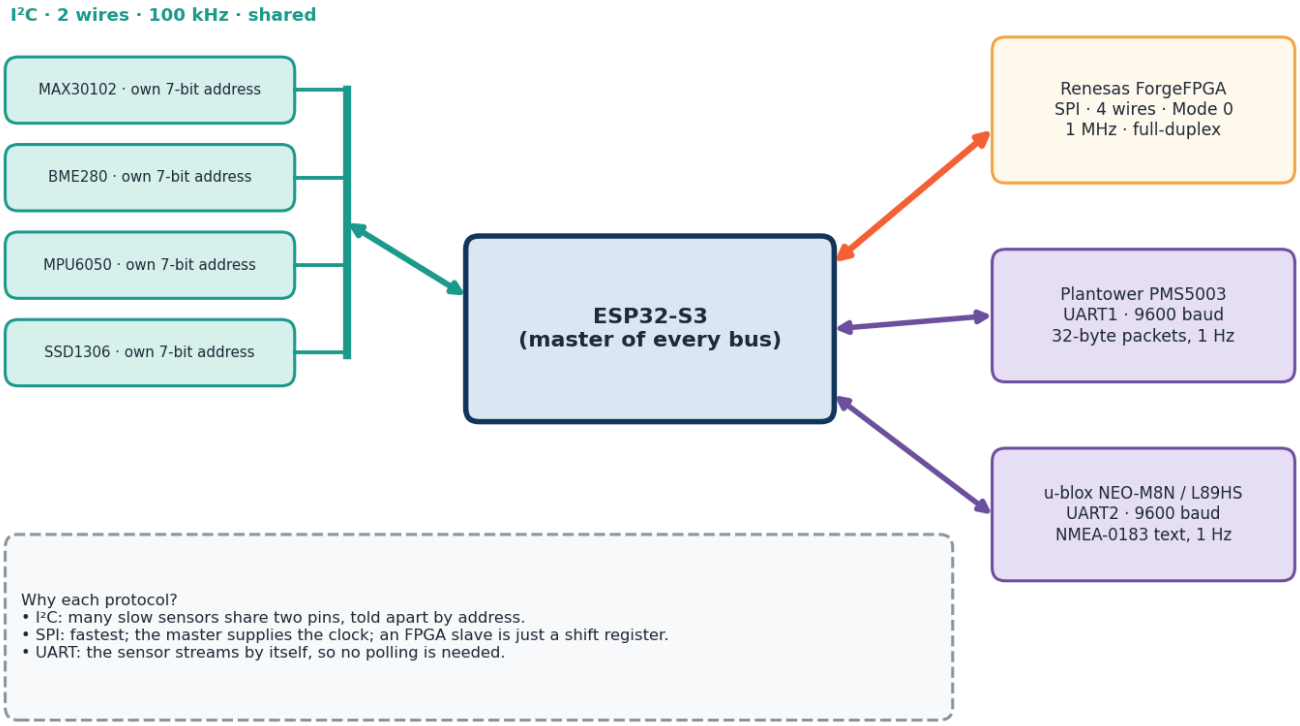



*Figure 6.1 — Three standard serial protocols connect the whole sensor network to the ESP32-S3.*

| Protocol | Connected devices | Bus type | Why it fits |
|---|---|---|---|
| **I²C** | MAX30102, BME280, MPU6050, SSD1306 | 2 wires, shared multi-drop | Several low-to-medium-speed sensors share two pins using distinct 7-bit addresses. |
| **SPI** | ForgeFPGA SLG47910 | 4 wires, dedicated point-to-point | Full duplex, minimal FPGA logic, deterministic cycle-by-cycle shifting. |
| **UART1** | Plantower PMS5003 | 2 wires, async | The sensor broadcasts 32-byte packets by itself; no polling commands needed. |
| **UART2** | u-blox NEO-M8N / Quectel L89HS | 2 wires, async | GNSS modules stream NMEA-0183 text lines. |

**IN ONE LINE**

I²C connects the onboard sensors over two shared wires, SPI links MCU to FPGA, and two UARTs receive the autonomous smoke and GPS streams.

**JUDGE MIGHT ASK**

**Q.** *Why four devices on one I²C bus?*

**A.** That is what I²C is for: unique addresses on two shared wires save MCU pins. Total bus utilisation is under 15 % at 100 kHz.

**Q.** *Why not read the MAX30102 over SPI?*

**A.** Common breakouts only expose I²C. At 100 Hz it uses less than 3 % of the bus.

**Q.** *Why UART for PMS5003 and GPS?*

**A.** Both are autonomous streamers. The ESP32's hardware UART FIFO buffers the stream with no master polling overhead.

PART II · HARDWARE AND RTL

# 7 FPGA and RTL Fundamentals Used in VALOR

**AFTER THIS CHAPTER YOU CAN…**

- ✓ explain the difference between running software and building hardware
- ✓ use the seven RTL concepts that appear in every VALOR module
- ✓ say why non-blocking assignment makes a real shift register

## 7.1 FPGA versus microcontroller

| | Microcontroller (software) | FPGA (hardware) |
|---|---|---|
| **What you write** | A program of instructions | A description of circuits |
| **Execution** | An ALU fetches, decodes and runs instructions one by one | Thousands of logic blocks run **in parallel, every clock** |
| **Timing** | An interrupt or higher-priority task delays everything (serial time) | Fixed by the clock: deterministic |
| **For VALOR** | Fusion, AI, networking, display | Filter + peak detector + timestamp |

When you synthesise Verilog you are not writing a program — **you are physically wiring dedicated digital circuits**. The filter and peak detector run continuously in silicon, completely independent of whatever the CPU is doing.

## 7.2 Seven RTL concepts, each shown in real VALOR code

### 1. Module — a self-contained circuit

```
module moving_average_8tap #(parameter DATA_WIDTH = 8)(
    input  wire                  clk,
    input  wire                  rst_n,
    input  wire                  data_valid,
    input  wire [DATA_WIDTH-1:0] data_in,
    output reg  [DATA_WIDTH-1:0] data_out,
    output reg                   out_valid );
```

*hardware/common/moving_average_8tap.v — the parameter lets the same filter serve 8-, 12- or 16-bit samples.*

### 2. Ports — the pins of a module

```
input  wire clk;        // 50 MHz system clock from the internal oscillator
input  wire spi_sck;    // shift clock from the ESP32-S3
input  wire spi_ss_n;   // active-low slave select
input  wire spi_mosi;   // serial data from the MCU
output wire spi_miso;   // serial data back to the MCU
output reg  led_user;   // heartbeat flash to the blue LED
```

*hardware/shrikefi/forgefpga_ppg_top.v — output reg means the pad is driven straight from a register.*

## 3. Registers (reg) — storage that holds a value between clocks

```
reg [1:0]  current_state, next_state;   // 2 flip-flops hold the FSM state
reg [DATA_WIDTH-1:0] prev_sample;
reg [DATA_WIDTH-1:0] peak_val;          // running maximum of the current rise
reg [31:0] refractory_cnt;              // 32-bit down counter
reg [31:0] interval_cnt;                // 32-bit up counter: cycles since last beat
```

*hardware/common/ppg_peak_detector.v*

## 4. Combinational versus sequential logic

```
// COMBINATIONAL: output follows inputs immediately, no memory
wire [DATA_WIDTH+2:0] next_sum = running_sum + {3'b000, data_in} - {3'b000, shift_reg[7]};

// SEQUENTIAL: updates only on a clock edge
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)          running_sum <= 11'd0;
    else if (data_valid) running_sum <= next_sum;
end
```

*shift_reg[7] holds x[n-8], the oldest of the eight taps (shift_reg[0] = x[n-1]).*

## 5. Always blocks — where behaviour is described

```
always @(*) begin                        // combinational: steering logic
    next_state = current_state;
    case (current_state)
        STATE_ARMED: if (sample_valid && sample_in >= dyn_threshold) next_state = STATE_RISING;
        ...
    endcase
end
always @(posedge clk or negedge rst_n)    // sequential: the state flip-flops
    if (!rst_n) current_state <= STATE_ARMED; else current_state <= next_state;
```

*The standard two-block FSM style: one block computes the next state, one block registers it.*

## 6. Blocking (=) versus non-blocking (<=)

Rule: **= for combinational logic, <= for clocked registers.**

```
shift_reg[0] <= data_in;
for (i = 1; i < 8; i = i + 1) shift_reg[i] <= shift_reg[i-1];
```

*Non-blocking assignments all read their right-hand sides first, then update together.*

**COMMON TRAP**

With blocking = here, data_in would cascade through all eight stages inside one clock edge and the pipeline would collapse into a single register. Non-blocking assignment is what makes shift_reg[1] receive the *old* shift_reg[0].

## 7. Synchronous reset generation

```
reg [7:0] por_cnt = 8'd0;
localparam [7:0] POR_LAST = 8'd255;
wire rst_n = (por_cnt == POR_LAST);
always @(posedge clk) if (por_cnt != POR_LAST) por_cnt <= por_cnt + 8'd1;
```

*For the first 255 clocks rst_n is LOW and holds every FSM, filter and shifter in reset; then it rises cleanly with no external reset pin.*

**IN ONE LINE**

Microcontrollers execute software one instruction at a time with OS jitter; FPGAs execute dedicated parallel circuits with deterministic clock timing.

**JUDGE MIGHT ASK**

**Q.** *Why non-blocking assignments in the shift register?*

**A.** They evaluate every right-hand side before updating any register, which models real flip-flops in a pipeline. Blocking assignments would race and collapse the pipeline.

**Q.** *wire versus reg?*

**A.** A wire is a continuous electrical connection and cannot store state. A reg holds a value assigned in an always block; in a clocked block synthesis turns it into flip-flops.

**Q.** *Why an FPGA when the ESP32-S3 runs at 240 MHz?*

**A.** The ESP32 runs FreeRTOS with preemption and 5 ms scheduling jitter, which corrupts beat-to-beat HRV. The FPGA detects peaks in hardware at 50 MHz with 20 ns timestamps, immune to CPU load.

PART II · HARDWARE AND RTL

# 8 The 8-Tap Moving Average Filter

**AFTER THIS CHAPTER YOU CAN…**

- ✓ write the filter equation and its running-sum form
- ✓ explain why ÷8 costs zero logic
- ✓ compute the register width needed to avoid overflow
- ✓ state the filter's delay and why it is acceptable

Raw photodiode signals carry thermal noise, 50/60 Hz light flicker and ADC quantisation ripple. If that ripple reaches the peak detector it makes false peaks. An 8-tap moving average smooths the signal first.

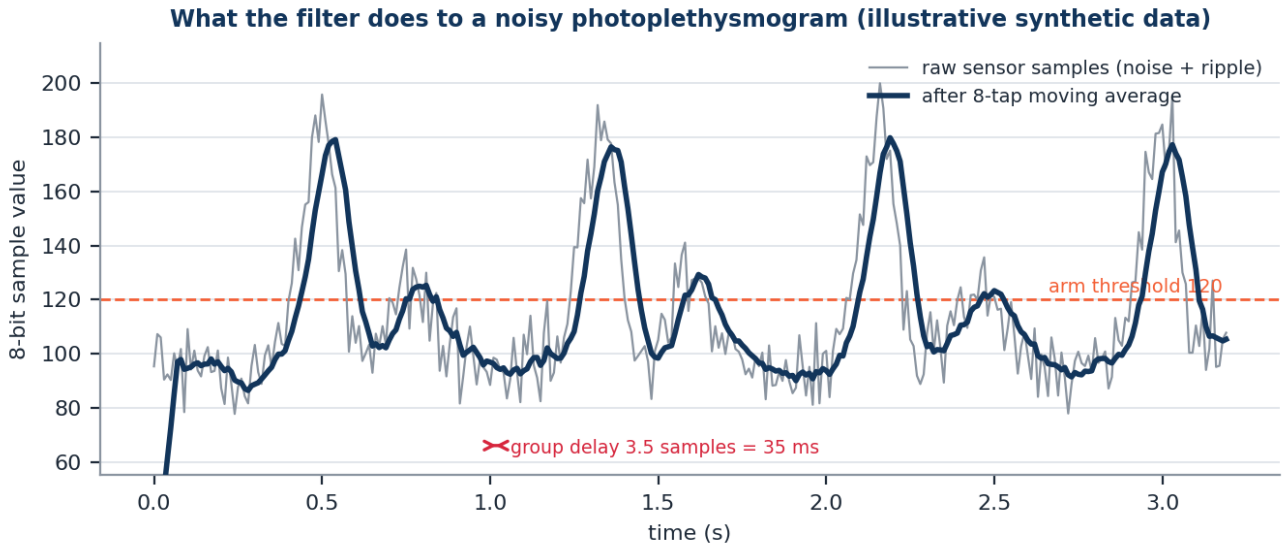



*Figure 8.1 — Raw samples (grey) versus the filtered output (navy). Synthetic data for illustration.*

## 8.1 The maths

$$y[n] = \tfrac{1}{8} \cdot \Sigma\, x[n-k] \quad k = 0 \ldots 7$$

Adding eight samples every clock would need an adder tree several logic levels deep. Instead the filter keeps a **running sum**: add the newest sample, subtract the one that just left the window.

$$\mathbf{Sum_{new} = Sum_{old} + x[n] - x[n-8]}$$

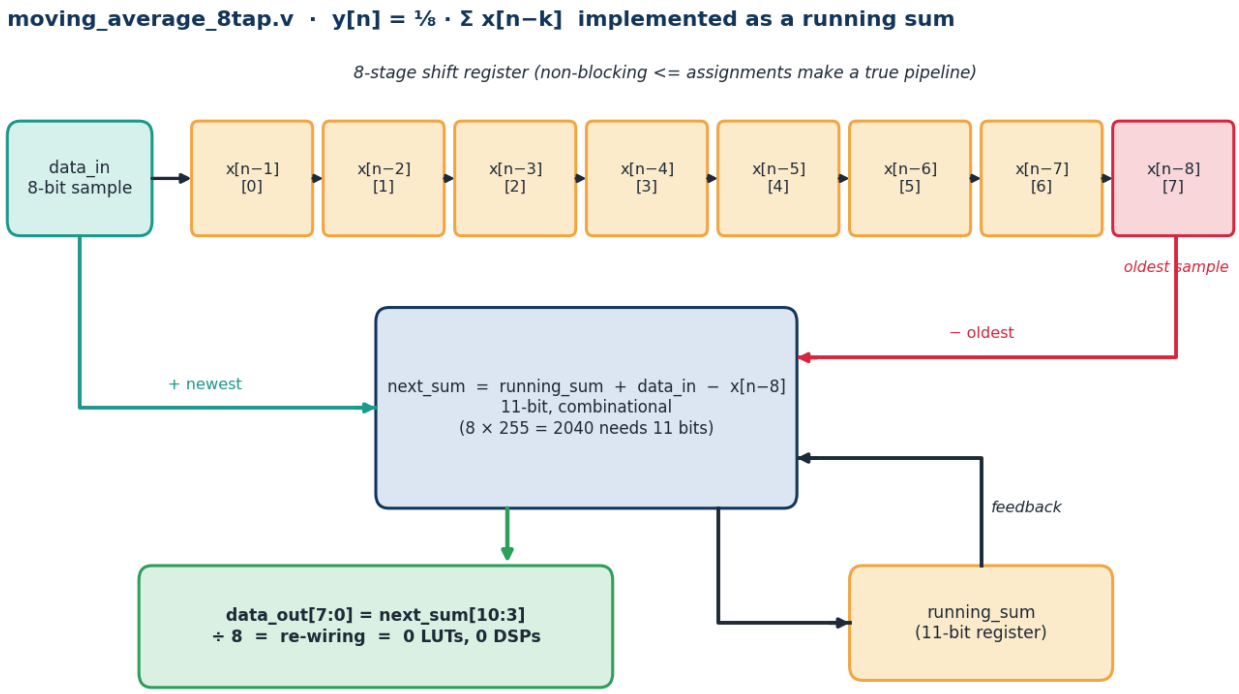



*Figure 8.2 — The filter datapath: a shift register remembers the eight samples, one adder-subtractor updates the sum, and the output is just re-wired.*

```
wire [DATA_WIDTH+2:0] next_sum = running_sum + {3'b000, data_in} - {3'b000, shift_reg[7]};
running_sum <= next_sum;
data_out    <= next_sum[DATA_WIDTH+2:3];     // 3-bit right shift = divide by 8
```

*Listing 8.1 — the core of moving_average_8tap.v*

## 8.2 Why dividing by 8 is free

Dividing by $2^k$ in binary is a right shift by k places. In an FPGA a fixed shift needs no gates at all — it is a choice of *which wires* connect to which. Bit 3 of next_sum goes to bit 0 of data_out, bit 4 to bit 1, and so on. **The division costs exactly zero LUTs.** Dividing by 7, in contrast, needs a real divider circuit.

## 8.3 Bit-width growth

**max sum = 8 × 255 = 2040 ⌈log₂ 2040⌉ = 11 bits**

`reg [DATA_WIDTH+2:0] running_sum` with DATA_WIDTH = 8 spans [10:0], exactly 11 bits. The extra 3 bits are guaranteed headroom, and dropping 3 bits on output returns the scale to 0–255.

## 8.4 Latency and cost

- **Group delay** of a symmetric FIR: (N − 1)/2 = 3.5 samples = **35 ms** at 100 Hz — imperceptible for heart-rate tracking.
- **Fill time:** 8 valid samples = 80 ms after boot.
- **Why 8 taps, not 16 or 32?** 8 taps removes ADC ripple and flicker with a small delay. Larger windows delay the signal further, flatten the sharp systolic crest, and double the flip-flop use on a 1,120-LUT part.

**IN ONE LINE**

An 11-bit running-sum register adds the newest sample, subtracts the oldest and divides by 8 with a 3-bit right shift that costs no logic.

**CHECK YOURSELF**

1) What would the filter output be for eight consecutive samples of 100? (Sum = 800; 800 ≫ 3 = 100.)

2) If DATA_WIDTH were 12, how wide must running_sum be? (DATA_WIDTH + 3 = 15 bits.)

**JUDGE MIGHT ASK**

**Q.** *Output width and why?*

**A.** 8 bits, matching the input. The 11-bit sum is shifted right 3 places so the output keeps the 0–255 range without losing dynamic range.

**Q.** *Why is ÷8 free but ÷7 expensive?*

**A.** Powers of two are wiring. ÷7 needs a sequential divider or iterative subtraction using dozens of LUTs.

PART II · HARDWARE AND RTL

# 9 The Peak Detector Finite State Machine

**AFTER THIS CHAPTER YOU CAN…**

- ✓ draw the four-state diagram and label every transition
- ✓ say why it is a registered Moore machine and why binary encoding was chosen
- ✓ explain the 25 % crest-fall rule and the 250 ms refractory period
- ✓ compute 12,500,000 cycles from first principles

`hardware/common/ppg_peak_detector.v` is the most critical circuit in the FPGA. It recognises a heartbeat in the filtered pulse wave and timestamps it. It does *not* look for a bit pattern like 10110; it recognises a **pulse morphology**: an upward crossing, a climb to a crest, a confirmed fall, then a mandatory blanking interval.

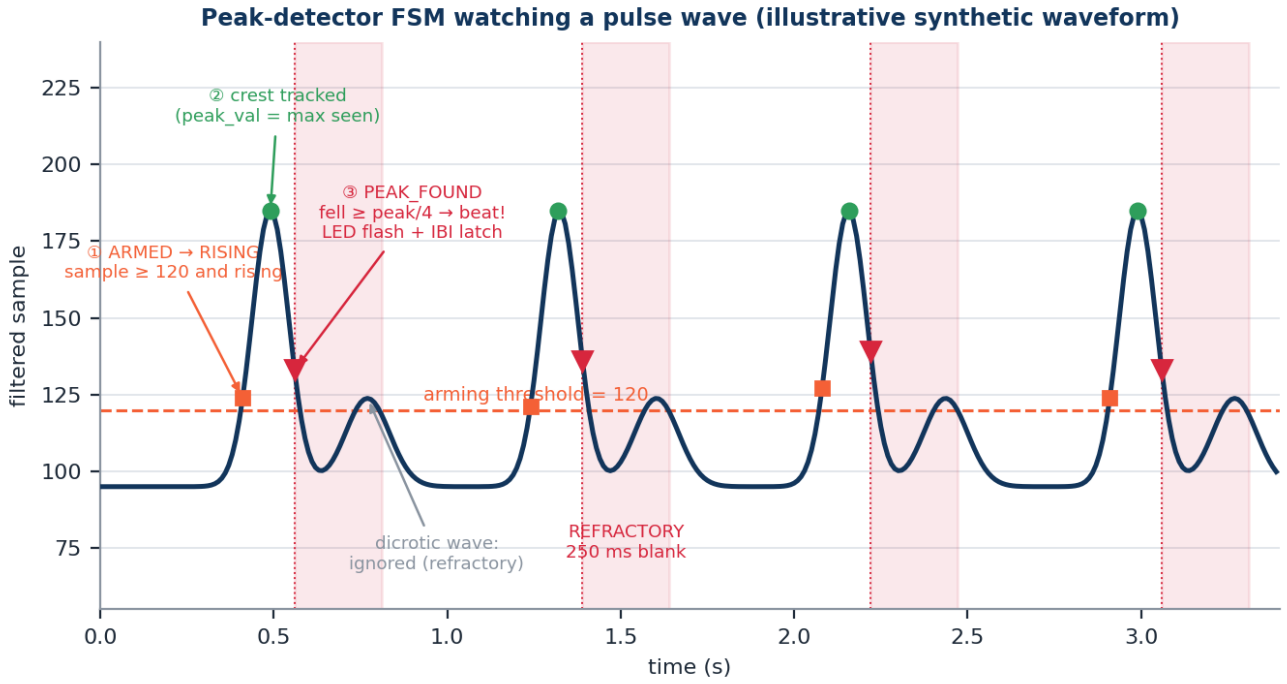



*Figure 9.1 — The FSM watching a synthetic pulse train. Squares = arming, circles = crest, triangles = beat confirmed, shaded = refractory.*

## 9.1 Moore, not Mealy

In a **Mealy** machine outputs depend on state *and* current inputs; in a **Moore** machine outputs depend on the state alone. VALOR's detector is a **registered Moore machine**: `beat_detected` is high only because the machine is *in* STATE_PEAK_FOUND, never because of what sample_in is right now. That guarantees a glitch-free one-cycle pulse.

```
case (current_state)
    STATE_ARMED:      beat_detected <= 1'b0;
    STATE_RISING:     beat_detected <= 1'b0;
    STATE_PEAK_FOUND: beat_detected <= (first_beat_seen) ? 1'b1 : 1'b0;
    STATE_REFRACTORY: beat_detected <= 1'b0;
endcase
```

*Listing 9.1 — outputs are assigned from the state only.*

## 9.2 States and encoding

```
localparam STATE_ARMED      = 2'b00;
localparam STATE_RISING     = 2'b01;
localparam STATE_PEAK_FOUND = 2'b10;
localparam STATE_REFRACTORY = 2'b11;
```

**Binary versus one-hot.** One-hot (0001, 0010, 0100, 1000) simplifies decode logic in large FPGAs with spare flip-flops. On a 1,120-LUT part, flip-flops are precious: binary uses **2** flip-flops, one-hot would use 4.

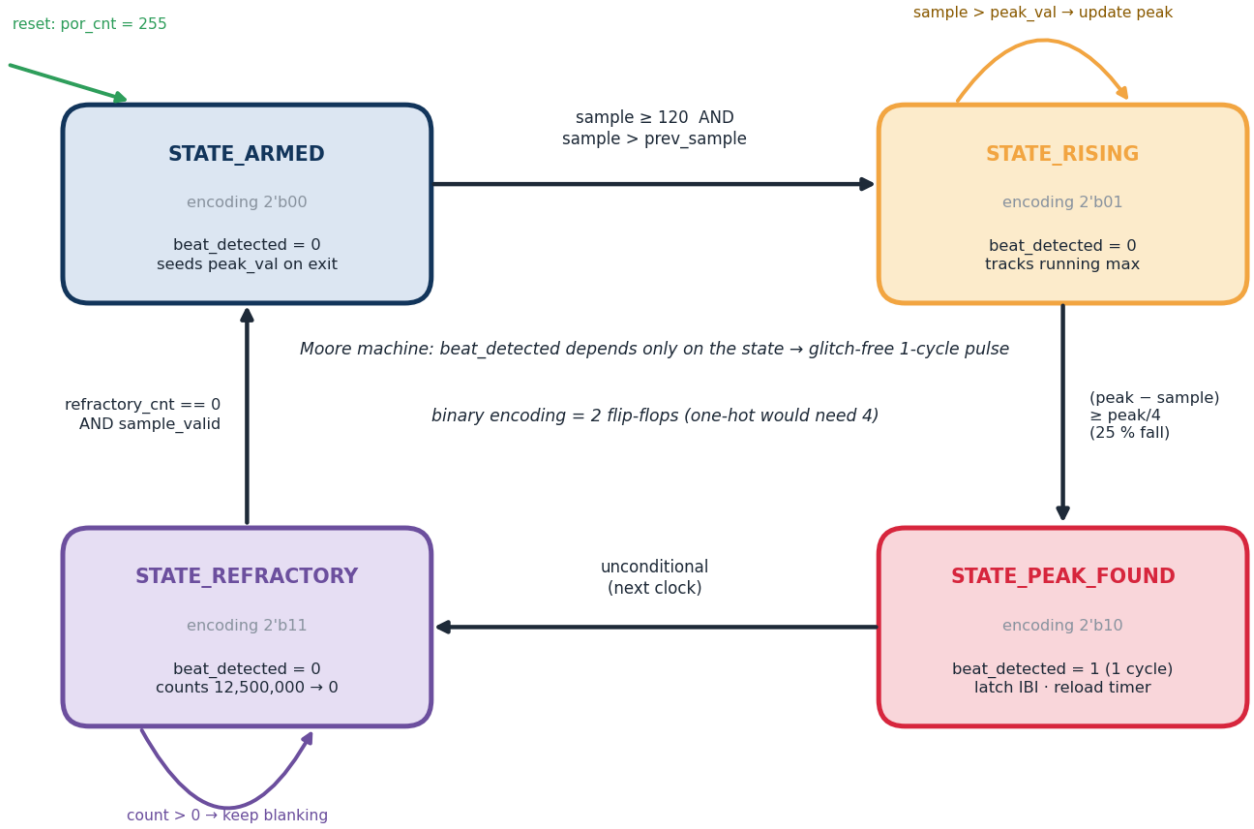



*Figure 9.2 — The state-transition diagram.*

## 9.3 What each state does

| State | Behaviour |
|---|---|
| ARMED 00 | Waits for a pulse. Moves to RISING only if **sample ≥ threshold (120)** *and* **sample > prev_sample**. The slope condition stops re-triggering on the slow decay of a wide pulse. Seeds peak_val. |
| RISING 01 | Tracks the highest value seen (peak_val). A true crest is confirmed when the signal drops below peak_val by at least `crest_fall_min`. |
| PEAK_FOUND 10 | Beat confirmed. Asserts beat_detected for one cycle (after the first beat), latches `ibi_cycles ≤ interval_cnt`, resets the interval counter, loads refractory_cnt = 12,500,000. Always goes to REFRACTORY next. |
| REFRACTORY 11 | Blanking window. Counts down every 50 MHz tick and ignores everything — dicrotic notch, reflected waves, noise. When the count is 0 and a valid sample arrives, returns to ARMED. |

```
wire [DATA_WIDTH-1:0] crest_fall_min = (peak_val > 8'd48) ? (peak_val >> 2) : 8'd6;
```

*Listing 9.2 — the proportional drop rule: a fall of 25 % of the peak for strong signals.*

> **ENGINEER'S NOTE**
>
> Samples are centred on 120 and arming needs sample ≥ 120, so peak_val is always ≥ 120 in this build. The 25 % rule is therefore always the active branch and the 6-count floor for weak signals is unreachable hardware. A judge who reads the RTL carefully may notice; it is better that you say it first.

**COMMON TRAP**

**Names in older notes.** An early deck described states IDLE, ARMED, CREST_FALL, REFRACTORY with a fixed drop of 3. The synthesisable Verilog is the only source of truth: ARMED, RISING, PEAK_FOUND, REFRACTORY with the proportional rule above.

| From | Condition | To | Action |
|---|---|---|---|
| ARMED | sample_valid ∧ sample ≥ thr ∧ sample > prev | RISING | peak_val ≤ sample; beat = 0 |
| ARMED | otherwise | ARMED | prev_sample ≤ sample |
| RISING | sample_valid ∧ (peak − sample) ≥ crest_fall_min | PEAK_FOUND | crest confirmed |
| RISING | sample_valid ∧ sample > peak_val | RISING | peak_val ≤ sample |
| PEAK_FOUND | unconditional | REFRACTORY | beat = 1; latch IBI; reload timer |
| REFRACTORY | refractory_cnt = 0 ∧ sample_valid | ARMED | otherwise keeps counting down |

## 9.4 The 250 ms refractory period

- **Why it exists.** The arterial pulse contains a secondary reflected wave, the *dicrotic notch* (aortic valve closure). Without blanking, the detector would count it as a second heartbeat and double the heart rate.
- **Ceiling it sets.** 60 s ÷ 0.250 s = **240 BPM**, enough for extreme tachycardia (200+) while rejecting bounces.
- **In clock cycles.** The RTL counts cycles, not milliseconds:

**50 MHz × 0.250 s = 12,500,000 cycles (parameter REFRACTORY_CYC = 12_500_000)**

## 9.5 From a beat to HRV: why 20 ns matters

The FPGA hands the ESP32 a cycle count for each interval. Dividing by 50,000 gives milliseconds (41,000,000 cycles = 820 ms). RMSSD is computed from successive IBI differences, so timing error in each IBI goes straight into the result.

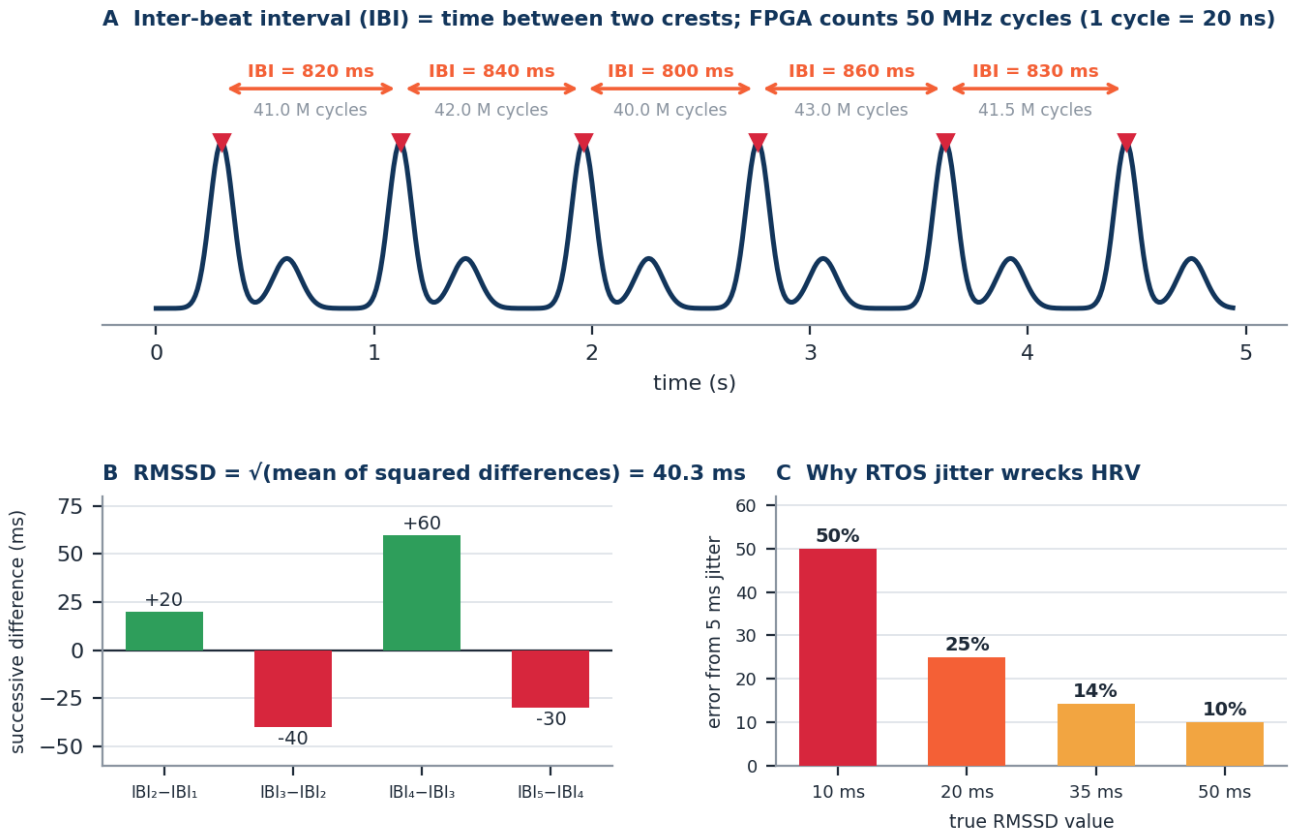



*Figure 9.3 — A: IBIs as cycle counts. B: a worked RMSSD from five beats. C: a fixed 5 ms RTOS jitter is a 10–50 % error against a typical RMSSD of 10–50 ms.*

**IN ONE LINE**

A registered 4-state Moore FSM with 2-bit binary encoding detects crests with a 25 % fall rule and then blanks for 250 ms (12.5 M cycles).

**JUDGE MIGHT ASK**

**Q.** *Moore or Mealy?*

**A.** Registered Moore: beat_detected is assigned in a clocked block purely from the state, so it cannot glitch with input noise.

**Q.** *Why four states?*

**A.** They are the minimum complete physiological cycle: wait (ARMED), track the rise (RISING), confirm and timestamp (PEAK_FOUND), blank the notch (REFRACTORY).

**Q.** *A noise spike arrives during the refractory window?*

**A.** Nothing happens; amplitudes are ignored and only the counter runs, giving 250 ms of hard noise immunity after each beat.

**Q.** *Is it synchronous?*

**A.** All transitions and counters change on posedge clk. Reset is asynchronous active-low (negedge rst_n), which gives immediate recovery; see Chapter 23 for the planned dual-flop reset.

PART II · HARDWARE AND RTL

# 10 The Other FSMs in the Design

**AFTER THIS CHAPTER YOU CAN…**

- ✓ describe the SOS and fall FSMs and their timers
- ✓ explain why a single threshold cannot detect a fall
- ✓ recognise the SPI shifter, burst sequencer and AXI handshake as small state machines

## 10.1 Emergency Assistance (SOS) latching FSM — sos.c

This governs how alarms escalate so the device does not “cry wolf”, and manages the full-screen emergency card. It is a Moore machine: the OLED card and buzzer depend only on the state.

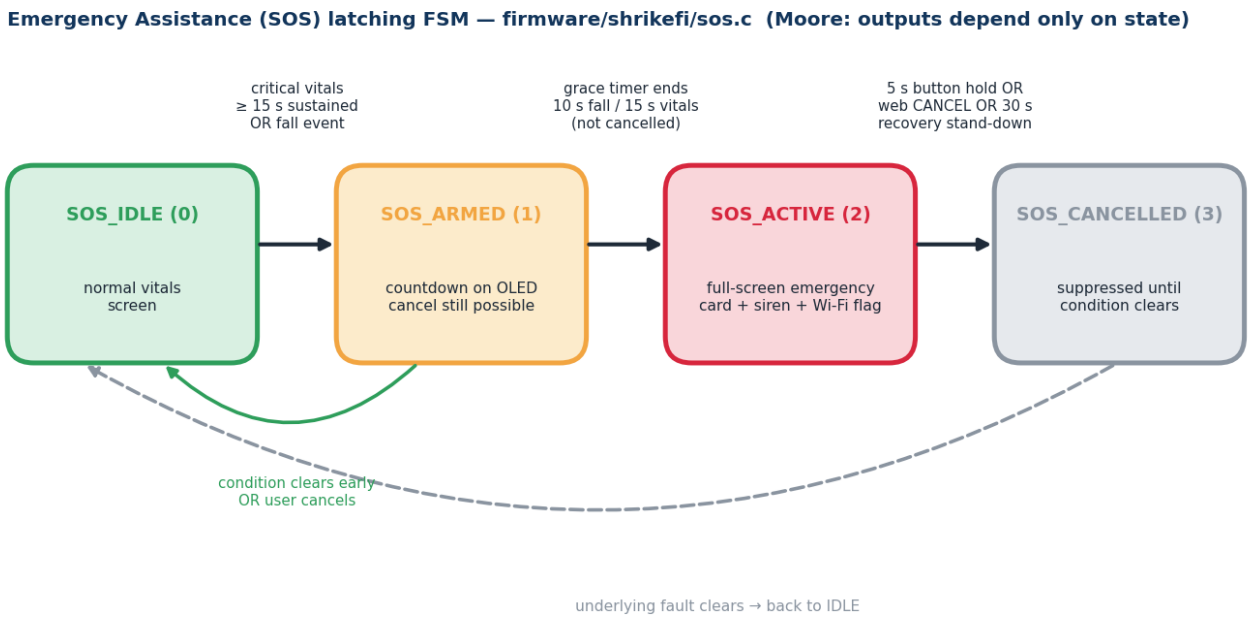


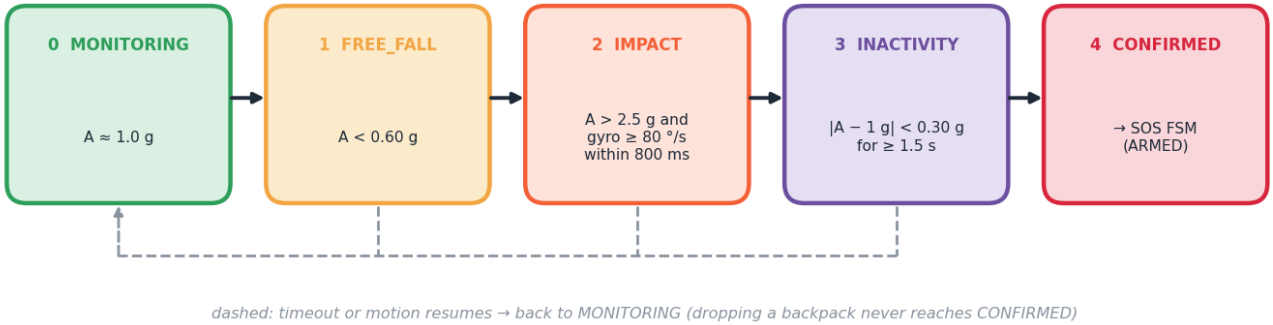

*Figure 10.1 — SOS escalation with grace windows (critical_confirm_ms = 15,000; fall_confirm_ms = 10,000).*

- **IDLE (0):** routine monitoring.
- **ARMED (1):** a critical condition or fall occurred. A grace timer runs — **15 s for critical vitals** (raised from 3 s to suppress motion false alarms) and **10 s for falls** — with an on-screen countdown so a conscious wearer can cancel.
- **ACTIVE (2):** latched. Full-screen OLED card, buzzer pattern, emergency flags over Wi-Fi.
- **CANCELLED (3):** operator cancelled (5-second button hold or web click) or 30 s of sustained recovery; stays suppressed until the underlying condition clears.

## 10.2 Fall-detection FSM — fall_detect.c

A fall has a signature sequence: **weightlessness → impact → stillness**. A single acceleration threshold cannot tell a fall from a dropped backpack; an FSM enforcing the whole sequence can.

**Fall-detection FSM (MPU6050, 40 Hz): free-fall → impact → immobility**



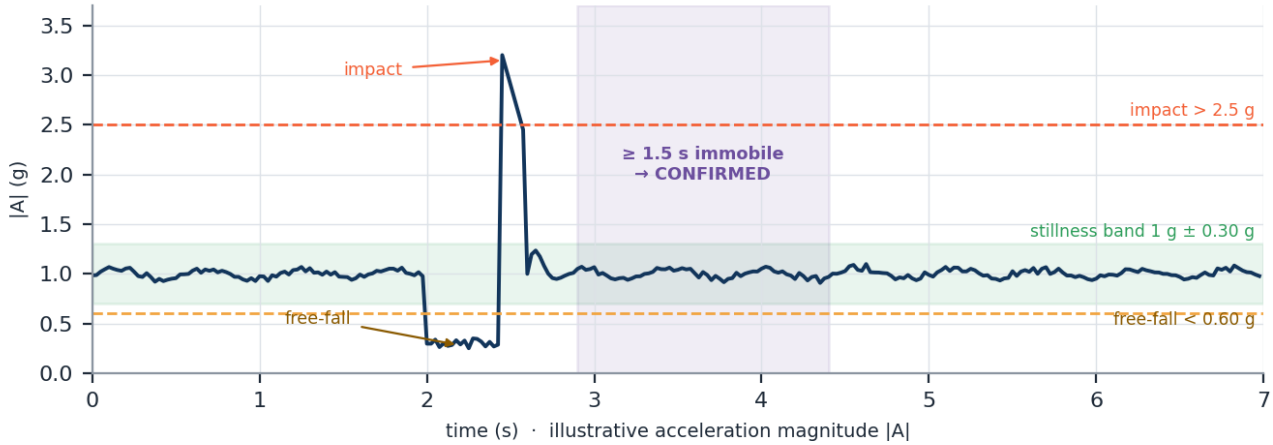




*Figure 10.2 — Five FSM states (three physical stages plus the idle and confirmed ends) and an illustrative acceleration trace.*

| State | Condition (A = acceleration magnitude) |
|---|---|
| 0 MONITORING | Baseline walking or sitting, A ≈ 1.0 g |
| 1 FREE_FALL | A < 0.60 g (weightlessness) |
| 2 IMPACT | Within 800 ms of free-fall: A > 2.50 g and angular rate ≥ 80 °/s |
| 3 INACTIVITY | \|A − 1 g\| < 0.30 g continuously for ≥ 1.5 s after impact |
| 4 CONFIRMED | Fall verified → hands over to the SOS FSM (ARMED) |

**ENGINEER'S NOTE**

You will see this described both as “three-stage” and “five-state”. Both are right: three physical stages (free-fall, impact, inactivity) sit between the MONITORING and CONFIRMED end states.

## 10.3 Three small hardware sequencers

| Block | File | What it is |
|---|---|---|
| SPI target shifter | spi_target.v | A shift register plus a 3-bit counter bit_cnt (0…7). When bit_cnt = 7 and a rising edge arrives it pulses o_rx_data_valid for one clock and resets. Not a multi-state FSM. |
| IBI burst sequencer | forgefpga_ppg_top.v | A Moore machine built from a 3-bit down-counter burst_cnt: 0 = normal {beat, filt[6:0]}; 5 = 0xC3; 4,3,2,1 = IBI bytes [31:24], [23:16], [15:8], [7:0]. |
| AXI4-Lite handshake (Zynq) | axi_ppg_accelerator.v | Decouples address, data and response phases: aw_done, w_done, write_execute, bvalid. Needed because the protocol lets address and data arrive on different cycles. |

**IN ONE LINE**

Beyond the peak detector, state machines gate fall kinematics, sequence SPI bursts, manage AXI handshakes and govern SOS escalation.

**JUDGE MIGHT ASK**

**Q.** *Why require an inactivity phase in fall detection?*

**A.** Jumping or dropping the device on a table causes a high deceleration, but the person keeps moving. Real incapacitation is impact followed by stillness.

**Q.** *What if the user recovers alone?*

**A.** The SOS FSM has a 30 s physiological recovery stand-down (recovery_clear_ms): normal, stable vitals for 30 continuous seconds clear the alarm.

**Q.** *Why not run fall detection on the FPGA?*

**A.** The IMU runs at 40 Hz, very slow for an FPGA, and the maths ($\sqrt{(Ax^2+Ay^2+Az^2)}$, angles) wants floating point, which the ESP32's FPU does easily but which would cost hundreds of LUTs.

PART II · HARDWARE AND RTL

# 11 Timing, Resources and Synthesis Results

**AFTER THIS CHAPTER YOU CAN…**

- ✓ explain synthesis versus place-and-route
- ✓ define slack, WNS and TNS and say why positive slack matters
- ✓ explain why CLB occupancy, not LUT count, is the binding constraint
- ✓ account for the growth from 342 to 515 LUTs

## 11.1 What the tools do

- **Synthesis** turns Verilog (if, case, +, <=) into a netlist of gates, multiplexers and flip-flops.
- **Place-and-route** assigns those cells to physical locations on the chip (*place*) and configures the routing switches that connect them (*route*).

## 11.2 Resource use of the production bitstream

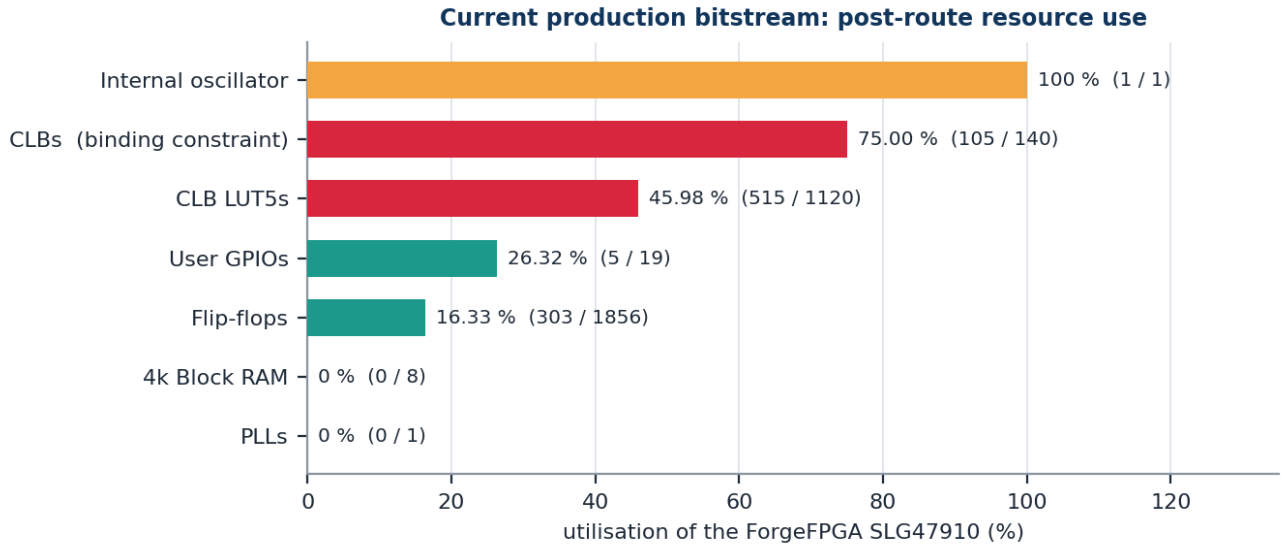



*Figure 11.1 — Post-route utilisation on the SLG47910 (report: resource_utilization_spi_link.log, top module forgefpga_ppg_top).*

| Resource | Used / available | % | Meaning |
|---|---|---|---|
| CLB LUT5s | 515 / 1120 | 45.98 | Filter, peak FSM and the IBI burst multiplexers; 605 LUTs still free. |
| Flip-flops | 303 / 1856 | 16.33 | 298 CLB flip-flops + 5 I/O flip-flops: pipeline data, 32-bit counters, synchronisers. |
| **CLBs** | **105 / 140** | **75.00** | **The true binding constraint.** Wide 32-bit counters consume CLBs even though 54 % of LUTs are unused. |
| User GPIOs | 5 / 19 | 26.32 | SCK, SS_n, MOSI, MISO, and the beat-stretched blue LED. |
| 4k Block RAM | 0 / 8 | 0 | None used; all 8 blocks free for future waveform buffers. |
| PLLs | 0 / 1 | 0 | Runs directly from the on-chip oscillator; the PLL is uncommitted. |
| Oscillator | 1 / 1 | 100 | Hard-wired 50 MHz relaxation oscillator; no external crystal. |

## 11.3 Why 342 became 515 LUTs

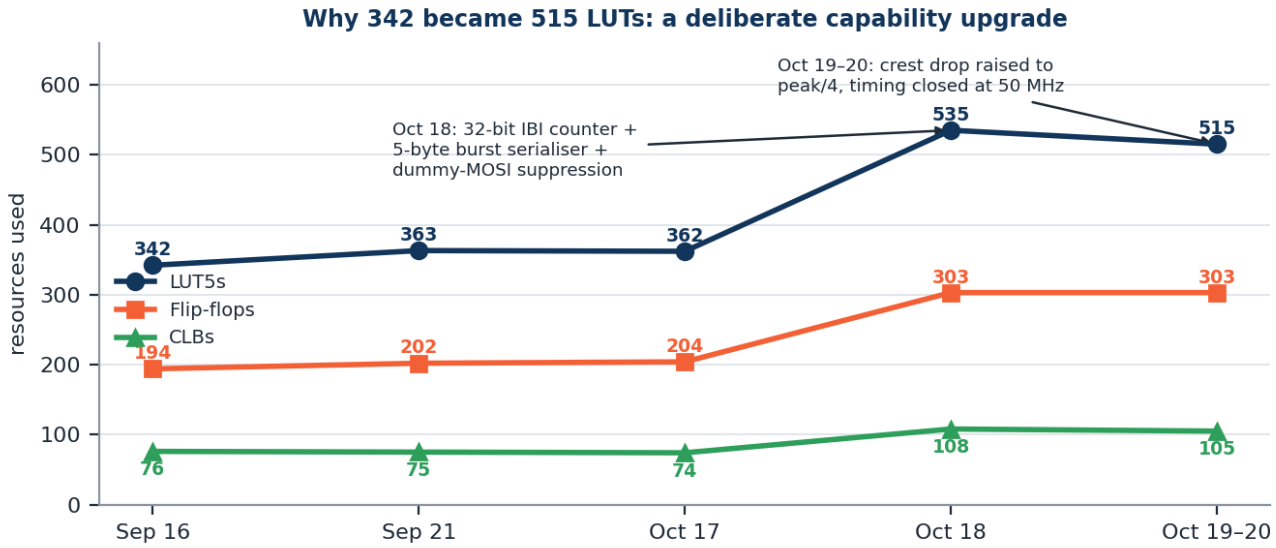



*Figure 11.2 — The history of the fit. The jump on 18 October was a deliberate capability upgrade, not bloat.*

- **Sep 16 — 342 LUTs / 194 FFs / 76 CLBs:** filter and simple threshold comparator, no cycle-accurate interval export.
- **Sep 21 — 363 / 202 / 75:** added a positive-slope comparator in ARMED to reject the pulse decay tail.
- **Oct 17 — 362 / 204 / 74:** clean re-fit of the pre-burst source.
- **Oct 18 — 535 / 303 / 108:** the 32-bit IBI counter (20 ns), the 5-byte 0xC3 burst serialiser, and **dummy-MOSI suppression** so the dummy bytes the ESP32 uses to clock out the burst never enter the filter.
- **Oct 19–20 — 515 / 303 / 105:** crest drop raised from peak/8 to **peak/4** to reject the dicrotic notch, and timing closed against the real 50 MHz SDC (bitstream 6930ba88…). This is the production bitstream.

## 11.4 Timing closure

**Slack** is the margin between when a signal must arrive and when it actually does.

$$\textbf{Slack} = T_{\textbf{required}} - T_{\textbf{arrival}}$$

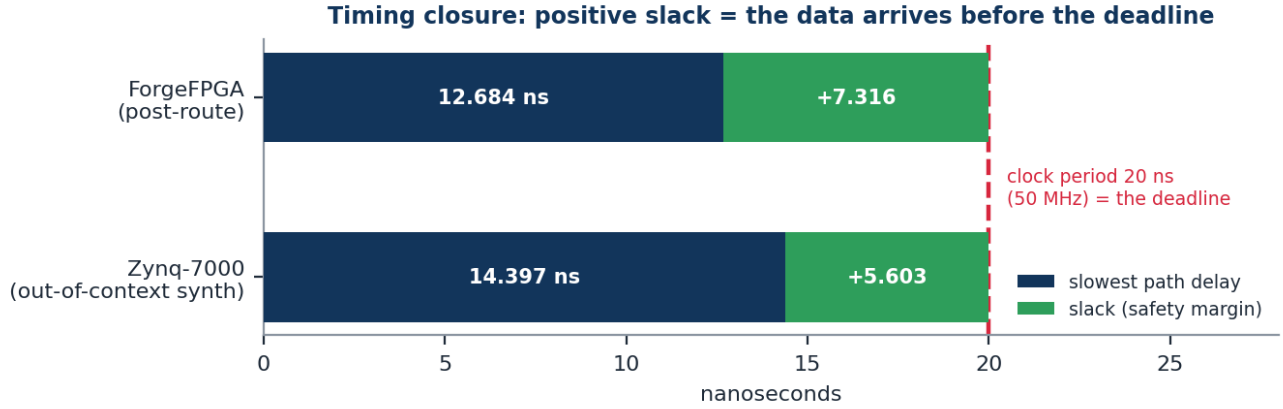



*Figure 11.3 — The 20 ns clock period is the deadline; slack is the safety margin left after the slowest path.*

| Target | Clock | WNS | Fmax | Other |
|---|---|---|---|---|
| **ForgeFPGA** (deployable) | 50 MHz internal oscillator, T = 20.000 ns (create_clock -period 20.000) | **+7.316 ns** | 78.85 MHz (12.683 ns) | TNS 0 ps over 1,260 endpoints |

| Target | Clock | WNS | Fmax | Other |
|---|---|---|---|---|
| **Zynq-7000** (golden) | 50 MHz AXI4-Lite bus clock | **+5.603 ns** | 69.45 MHz | WHS +0.184 ns; full block design in Vivado 2026.1: WNS +13.168 ns |

Worst Negative Slack (WNS) is the margin of the slowest path in the whole design. If it were negative the receiving flip-flop could capture a half-settled value, causing intermittent failures. Positive slack of 5–7 ns means the slowest calculation ends that much before the next clock edge, with margin for temperature, process and voltage variation.

**IN ONE LINE**

515/1120 LUT5s (45.98 %) and 105/140 CLBs (75.00 %), 0 DSP and 0 BRAM, closed 50 MHz timing with WNS +7.316 ns (Fmax 78.85 MHz) on ForgeFPGA and +5.603 ns on Zynq.

**COMMON TRAP**

Never say “the FPGA is out of logic” or “the FPGA has no BRAM”. The chip has 8 BRAMs and under 46 % LUT use; CLB routing density is what limits the floorplan.

**JUDGE MIGHT ASK**

**Q.** *Why did LUTs rise from 342 to 515?*

**A.** We deliberately added the 32-bit interval counter, the 5-byte burst serialiser and dummy-MOSI suppression to give the MCU 20 ns timestamps for HRV.

**Q.** *Why is CLB occupancy 75 % when LUT use is 46 %?*

**A.** Each CLB holds 8 LUT5s and flip-flops. Wide 32-bit counters and shift registers span CLB slices, so CLBs are allocated even when some LUTs inside stay unmapped.

**Q.** *What is your timing margin on ForgeFPGA?*

**A.** Against a real 20.0 ns SDC clock the post-route WNS is +7.316 ns with zero failing endpoints; achievable Fmax is 78.85 MHz.

**Q.** *Why is BRAM zero on both targets?*

**A.** An 8-bit pipeline and a 4-state FSM synthesise into distributed flip-flops, which keeps power tiny and leaves every SRAM block free.

PART III · SOFTWARE AND INTELLIGENCE

# 12 The ESP32-S3 Firmware

**AFTER THIS CHAPTER YOU CAN…**

- ✓ explain why tasks are pinned to cores
- ✓ list the six tasks, their priorities, stacks and rates
- ✓ explain the mutex and the torn-read problem
- ✓ describe the SoftAP dashboard flow

The firmware is C11 on Espressif ESP-IDF v5.5.x, running under FreeRTOS. FreeRTOS uses **preemptive, priority-based scheduling**: a higher-priority task immediately interrupts a lower-priority one. By pinning jitter-critical sampling to Core 0 and heavy analysis to Core 1, a long calculation or a burst of Wi-Fi traffic can never delay a 100 Hz optical read.

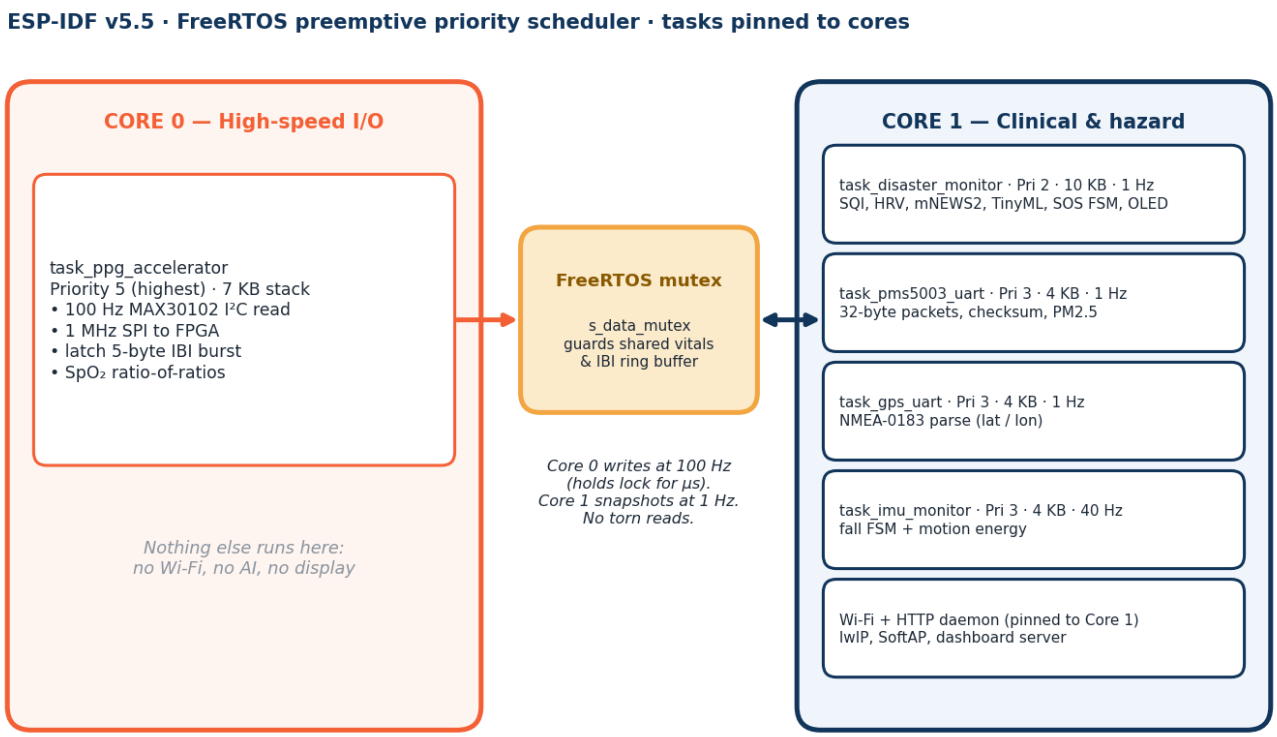



*Figure 12.1 — Task allocation across the two cores, joined only by a mutex-protected shared structure.*

| Task | Core | Pri | Stack | Rate | Job |
|---|---|---|---|---|---|
| task_ppg_accelerator | 0 | 5 | 7 KB | 100 Hz | MAX30102 I²C read, SPI exchange with the FPGA, latch IBI burst, $SpO_2$ |
| task_disaster_monitor | 1 | 2 | 10 KB | 1 Hz | Master fusion loop: snapshot vitals, BME280, Karlen SQI, pressure regression, INT8 inference, mNEWS2, SOS FSM, OLED |
| task_pms5003_uart | 1 | 3 | 4 KB | 1 Hz | 32-byte packets, checksum, PM2.5 into the rolling calibration window |
| task_gps_uart | 1 | 3 | 4 KB | 1 Hz | NMEA-0183 sentences → latitude and longitude |
| task_imu_monitor | 1 | 3 | 4 KB | 40 Hz | MPU6050 polling, fall FSM, dynamic motion energy |
| Wi-Fi + HTTP daemon | 1 | — | — | — | lwIP stack and web server, pinned with CONFIG_ESP_WIFI_TASK_PINNED_TO_CORE_1=y |

## 12.1 Why a mutex?

Core 0 writes new vitals at 100 Hz; Core 1 reads them at 1 Hz. A 32-bit float is not guaranteed to be written atomically relative to a read on the other core: without a lock, Core 1 could read half of a value that Core 0 is part-way through writing — a **torn read** — and corrupt a clinical number. Core 0 therefore holds `s_data_mutex` only for microseconds; Core 1 takes a local snapshot and then works on its own copy.

## 12.2 Sensor drivers in one paragraph each

- **MAX30102:** I²C; Red and IR LEDs pulse alternately; reflected light is read from the internal 32-sample FIFO. Writing 0x40 to INT_ENABLE_1 enables PPG_RDY (bit 6, per-sample data-ready) rather than A_FULL (bit 7).
- **BME280:** I²C; reads factory trimming coefficients at boot, then returns temperature-compensated pressure and humidity.
- **PMS5003:** UART1, 9600 baud; the driver hunts for the start bytes, validates a 16-bit checksum and extracts particle counts.
- **SSD1306:** I²C; text and emergency boxes are drawn into a 1,024-byte in-memory buffer (128 × 64 bits) and flushed at 1 Hz.
- **I²C robustness:** the HAL (esp32_i2c_hal.c) aborts any transfer over 50 ms and resets the controller instead of hanging the task.

## 12.3 The offline Wi-Fi SoftAP and dashboard

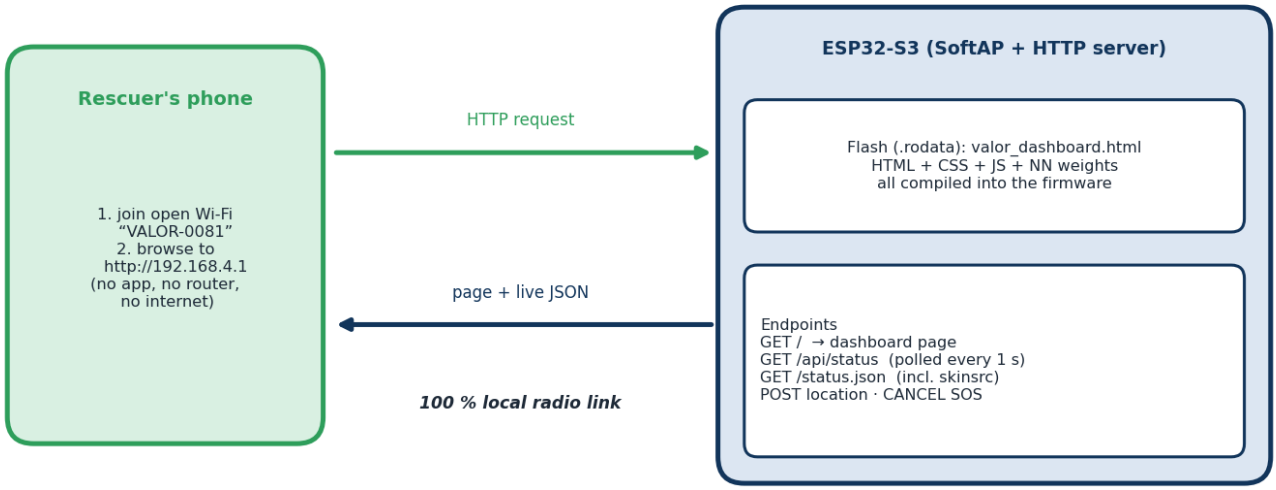



*Figure 12.2 — The wearable is its own web server. No router, no app, no internet.*

- The ESP32-S3 runs an **open** hotspot, SSID `VALOR-0081`, at 192.168.4.1.
- `valor_dashboard.html` is compiled into the firmware as read-only data. The phone's browser runs the page's JavaScript, which polls `/api/status` every second to animate gauges, advisories and 24-hour trends.

**IN ONE LINE**

FreeRTOS dedicates Core 0 to 100 Hz FPGA-assisted optical sampling, while Core 1 does triage, fusion, AI, display and the offline web server.

**JUDGE MIGHT ASK**

**Q.** *Why pin tasks instead of letting FreeRTOS choose?*

**A.** Migration causes cache thrashing and timing unpredictability. Pinning optical sampling to Core 0 means Wi-Fi traffic or inference on Core 1 can never drop a sample.

**Q.** *What if the I²C bus locks up?*

**A.** The HAL times out after 50 ms, resets the controller and returns an error code; the task keeps running.

**Q.** *Why a mutex on shared vitals?*

**A.** To prevent torn reads between a 100 Hz writer and a 1 Hz reader on different cores.

**Q.** *What if a task overflows its stack?*

**A.** Stack canary checking (CONFIG_FREERTOS_CHECK_STACK_OVERFLOW) triggers an identifiable abort. We profiled usage and gave 10 KB to the monitor task and 7 KB to the accelerator task. FreeRTOS stacks do not grow automatically.

PART III · SOFTWARE AND INTELLIGENCE

# 13 The AI Layer (TinyML)

**AFTER THIS CHAPTER YOU CAN…**

- ✓ justify running AI on the device
- ✓ explain an MLP, ReLU and INT8 quantisation in plain words
- ✓ reproduce the 619-parameter count
- ✓ describe what the 94.11 % MIMIC-III figure does and does not mean

## 13.1 Why the AI runs on the wearable

- **Offline survival** — there is no cellular infrastructure in an active disaster zone.
- **Speed** — a forward pass takes tens of microseconds (about 20–50 µs on the ESP32-S3; 0.44 µs on an x86 host) versus 200–2,000 ms for a cloud round trip, leaving over 99.9 % of the CPU free in each 1-second cycle.
- **Privacy** — heart rhythm and vitals never leave the body.
- **Energy** — inference costs microjoules; continuous radio transmission drains the battery.

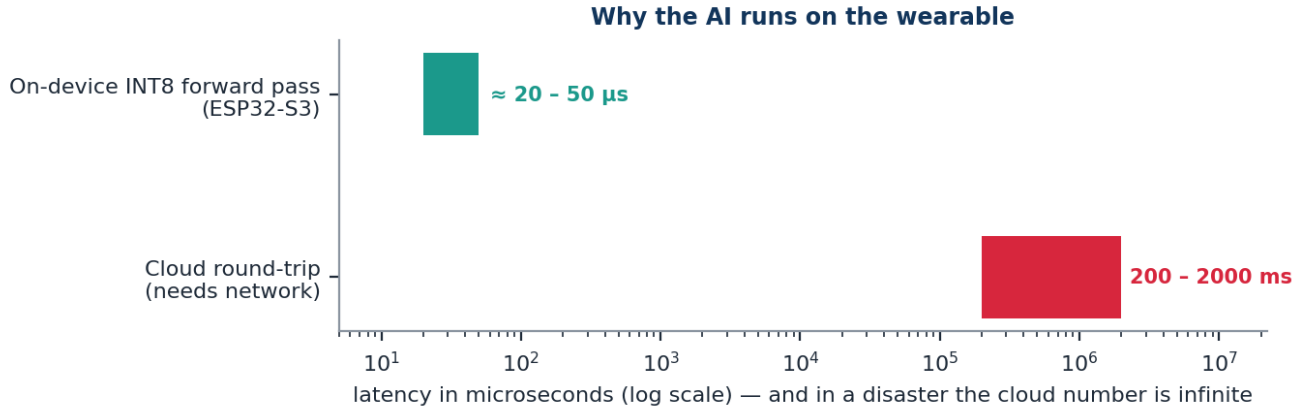



*Figure 13.1 — Latency of local inference versus a cloud round trip (log scale).*

**ENGINEER'S NOTE**

Clock speeds appear as both 240 MHz (what the ESP32-S3 can run) and 160 MHz (the configuration the 20–50 µs figure refers to). Quote the latency with the 160 MHz setting.

## 13.2 What a neural network is, in plain words

**ANALOGY**

An experienced triage doctor does not read heart rate in isolation; they weigh it against temperature, oxygen and breathing. A multilayer perceptron (MLP) does that arithmetically.

- **Inputs** — the raw measurements.
- **Weights** — numbers for how strongly each input influences each node, learned in training.
- **Hidden layers** — nodes that combine inputs into intermediate ideas such as “heat strain”.
- **ReLU** — f(x) = max(0, x): a filter that lets only meaningful stress through and makes the network non-linear.
- **Outputs** — three risk scores between 0.0 and 1.0 (sigmoid).

## 13.3 The multi-hazard risk model: 6 → 24 → 16 → 3

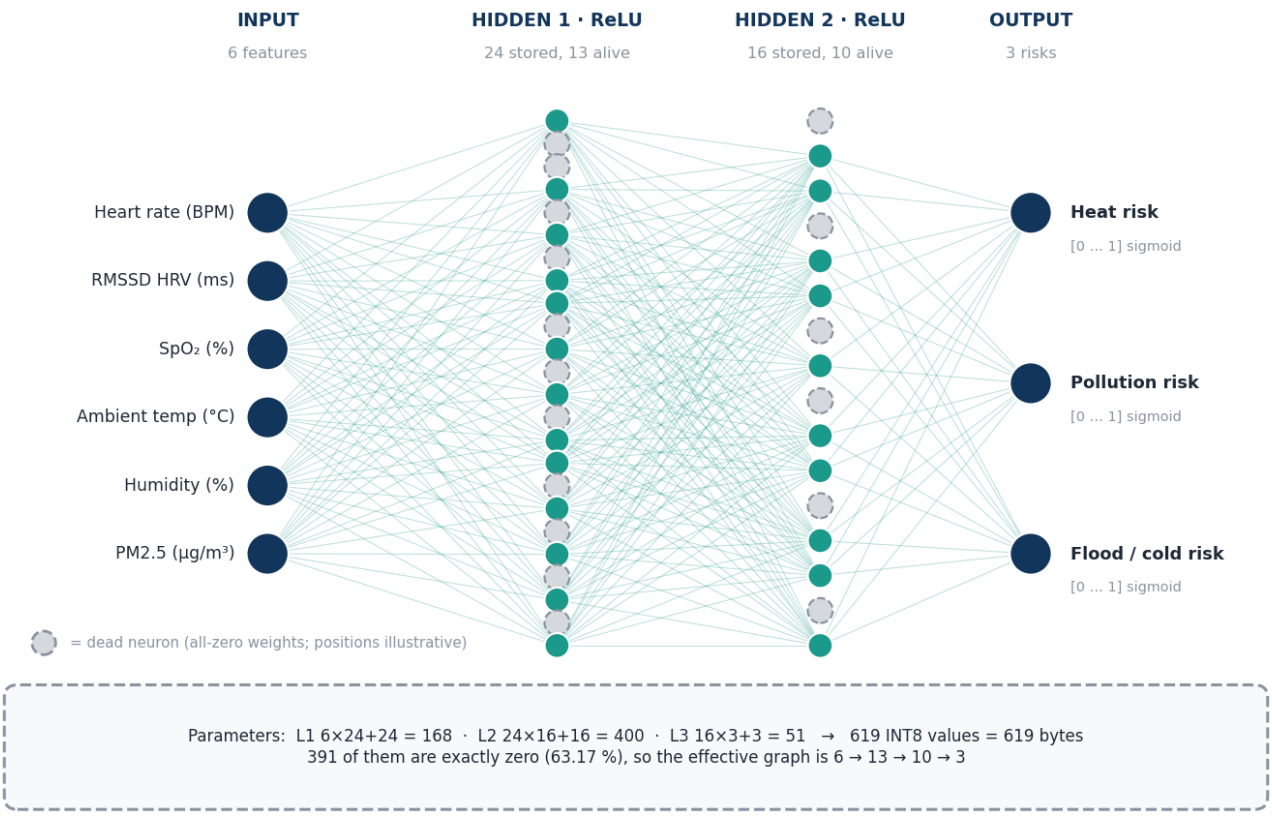



*Figure 13.2 — The stored network and its effective, pruned form.*

| Layer | Weights | Biases | Parameters |
|---|---|---|---|
| 1: 6 inputs × 24 neurons | 144 | 24 | 168 |
| 2: 24 × 16 | 384 | 16 | 400 |
| 3: 16 × 3 | 48 | 3 | 51 |
| **Total** | 576 | 43 | **619** → 619 bytes as INT8 |

**HONEST DISCLOSURE**

**Sparsity audit (from firmware/core/nn_risk_model_int8.c.inc).** 391 of the 619 stored values (63.17 %) are exactly zero. 11 of 24 layer-1 neurons and 6 of 16 layer-2 neurons are dead (all-zero weights with negative bias, so ReLU always outputs 0). The model behaves as an effective **6 → 13 → 10 → 3** network with 228 non-zero parameters. The 619-byte footprint counts stored values; the working compute graph is smaller. Saying this first shows you understand your own model.

## 13.4 The deployed kernel: dequantised weights on the FPU

- Six inputs (HR, RMSSD, $SpO_2$, temperature, humidity, PM2.5) are min-max normalised to [0, 1] in floating point.
- The 619 weights are stored as signed 8-bit integers. At inference each is dequantised on the fly with a per-layer scale S:

$$w_{float} = w_{int8} \times S$$

- Maths runs in float32 on the ESP32-S3's hardware FPU (576 multiply-accumulates) and the 3 sigmoids call `expf()`. This avoids the rounding and saturation problems of integer-only fixed-point.
- Older planning notes describe an integer-only kernel with a 32-step sigmoid lookup table. That belonged to the retired PM2.5 calibration network; the shipped hazard model uses float arithmetic with INT8-stored parameters (AI_MODEL_AUDIT_2026-10-20.md).

## 13.5 Quantisation: what it costs

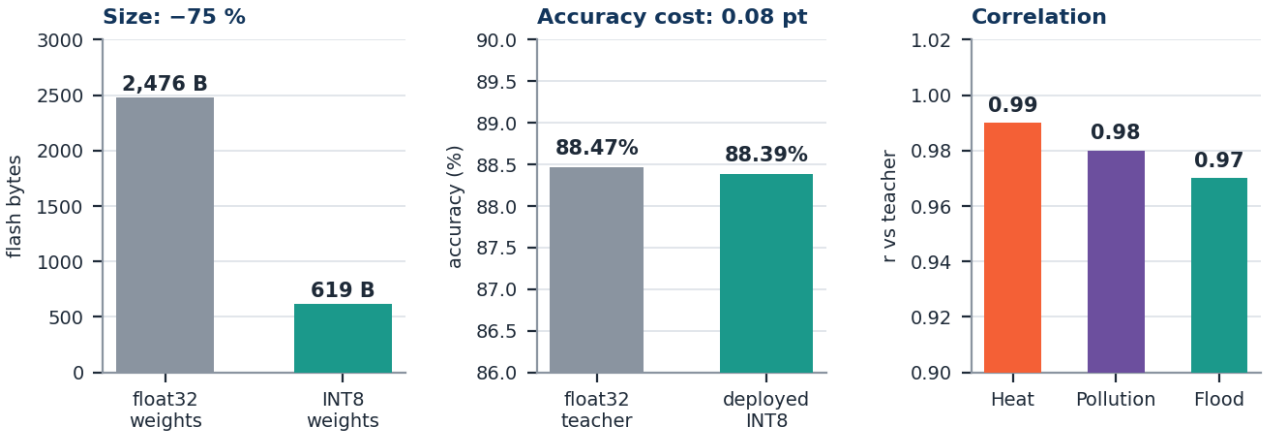



*Figure 13.3 — 75 % smaller, 0.08 percentage points less accurate, strongly correlated with the teacher model.*

- **Footprint:** 619 bytes versus 2,476 bytes for float32 weights.
- **Accuracy** on 7,000 held-out samples: 88.39 % deployed INT8 versus 88.47 % float32 teacher.
- **Risk-tier agreement:** 97.32 % between float32 and INT8; mean absolute error 0.00790, maximum 0.09136.
- **Latency:** about 20–50 µs on the ESP32-S3 (576 float MACs + 3 sigmoids).

## 13.6 The PM2.5 neural calibrator we removed — and why

An early design used a small 3 → 8 → 4 → 1 network (raw PM2.5, temperature, humidity → corrected mass) to calibrate the PMS5003 laser sensor. On the bench on 2026-10-20 it **cut readings by 31–53 % in dry air and collapsed small readings to zero at high humidity**. In a smoke-filled disaster zone, telling a responder that hazardous air is clean is a fatal failure mode.

We deleted it and replaced it with the published US EPA / AirGradient banded polynomial equations (`pm25_correction.c`) using a 60-second rolling average and explicit confidence flags (HUMID, DUST, SETTLING). **Lesson:** a model you cannot validate is worse than a transparent equation, even if the model sounds more impressive.

## 13.7 MIMIC-III verification

MIMIC-III v1.4 (MIT and Beth Israel Deaconess) is an anonymised ICU database. Using `mimic_harness.c` and `accuracy_evaluator.c` we ran the full clinical triage engine over **16,387 ICU vital-sign records from 98 patients**.

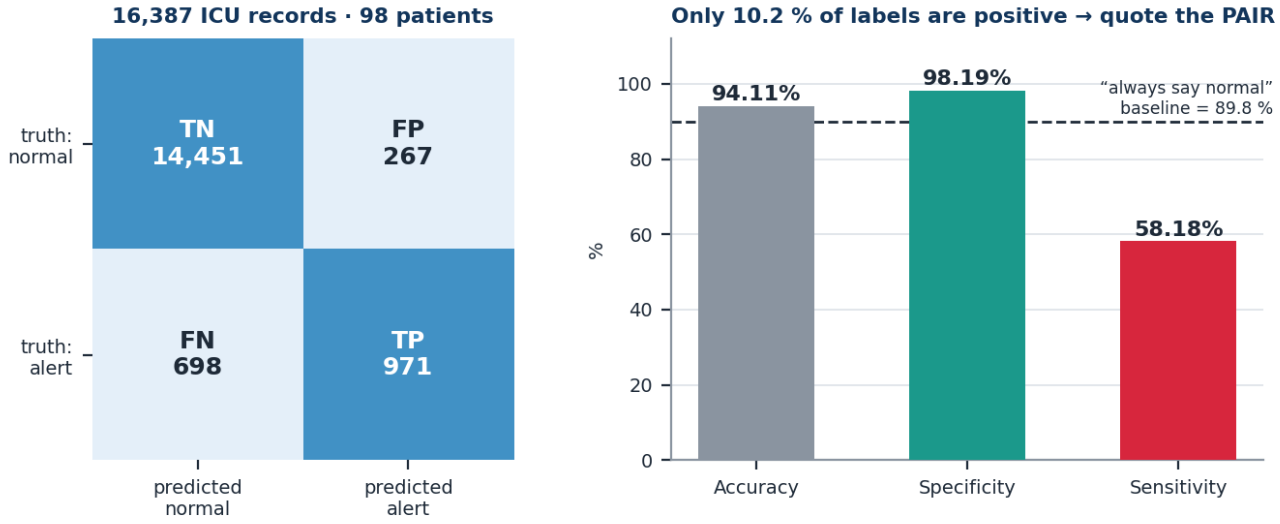



*Figure 13.4 — Confusion matrix and the three metrics. Accuracy alone hides the weak sensitivity.*

| Metric | Value | Calculation |
|---|---|---|
| Accuracy | 94.11 % | (14,451 + 971) / 16,387 |
| Specificity | 98.19 % | 14,451 TN / (14,451 + 267 FP) |
| Sensitivity | 58.18 % | 971 TP / (971 + 698 FN) |
| FP32 ↔ INT8 decision agreement | 99.95 % | 16,379 of 16,387 identical |

**HONEST DISCLOSURE**

**What 94.11 % means:** our embedded pipeline and quantised model faithfully reproduce our clinical triage rules on tens of thousands of real ICU records.

**What it does not mean:** it is not a prospective clinical trial, and the device does not diagnose disease. The ground-truth labels came from an algorithmic rule, so this proves *agreement with our rule*, not with a doctor.

**Why you must quote the pair:** only 10.2 % of labels are positive, so a lazy classifier that always says "normal" would already score 89.8 %. The honest summary is 58.18 % sensitivity / 98.19 % specificity.

**Data caveat:** heart rate and $SpO_2$ are real chart recordings; the RMSSD (HRV) column is **synthetic**, because MIMIC-III v1.4 stores periodic summaries, not high-frequency waveforms.

**IN ONE LINE**

A 619-byte INT8 MLP (6 → 24 → 16 → 3) runs in tens of microseconds on the ESP32-S3 FPU: 88.39 % against its teacher on 7,000 samples and 94.11 % triage agreement across 16,387 MIMIC-III records.

**JUDGE MIGHT ASK**

**Q.** *Why not a CNN or Transformer?*

**A.** A wearable must run on milliwatts. A 619-byte MLP does the job in tens of microseconds; larger models would need megabytes of RAM and drain the battery for negligible gain on a six-number input.

**Q.** *Is this a clinical diagnosis?*

**A.** No. It is hazard-risk triage. Absolute emergency decisions belong to deterministic mNEWS2 rules.

**Q.** *How did you check accuracy?*

**A.** A C harness over 16,387 records from 98 patients: 94.11 % triage agreement and 99.95 % float-to-INT8 agreement.

**Q.** *What is overfitting and did you check?*

**A.** Memorising training noise and failing on new patients. We used a held-out split with early stopping, kept capacity tiny, and wrapped outputs in deterministic safety rules.

PART III · SOFTWARE AND INTELLIGENCE

# 14 Clinical and Disaster Logic (the Rule-Based Parts)

**AFTER THIS CHAPTER YOU CAN…**

- ✓ score a patient with mNEWS2 tiers
- ✓ state the cyclone, heat and smoke thresholds
- ✓ explain the hybrid safety envelope and why rules beat the AI

## 14.1 mNEWS2

The UK Royal College of Physicians' National Early Warning Score 2 detects deteriorating hospital patients. VALOR implements a modified version using the four vitals a wearable can measure: **pulse, $SpO_2$, respiration rate and temperature** (0–3 points each).

| Score | Meaning | Response |
|---|---|---|
| 0 – 4 | Low risk | Routine monitoring |
| 5 – 6 | Medium risk | Urgent review threshold |
| ≥ 7 | High / critical | Immediate emergency response |
| any single 3 | Single-parameter red flag (e.g. $SpO_2 \leq 91$ %) | Escalates straight to high risk |

**HONEST DISCLOSURE**

Real NEWS2 also scores consciousness (AVPU) and supplemental oxygen, which need a human. VALOR marks consciousness as alert unless a fall is confirmed. The temperature row is deliberately withheld at 0 — see Chapter 15.

## 14.2 Cyclone and storm engine — pressure_trend.c

Absolute pressure depends on altitude and season, so a fixed threshold is useless. VALOR fits a **least-squares line over a 180-minute rolling window** and watches the *rate of fall*.

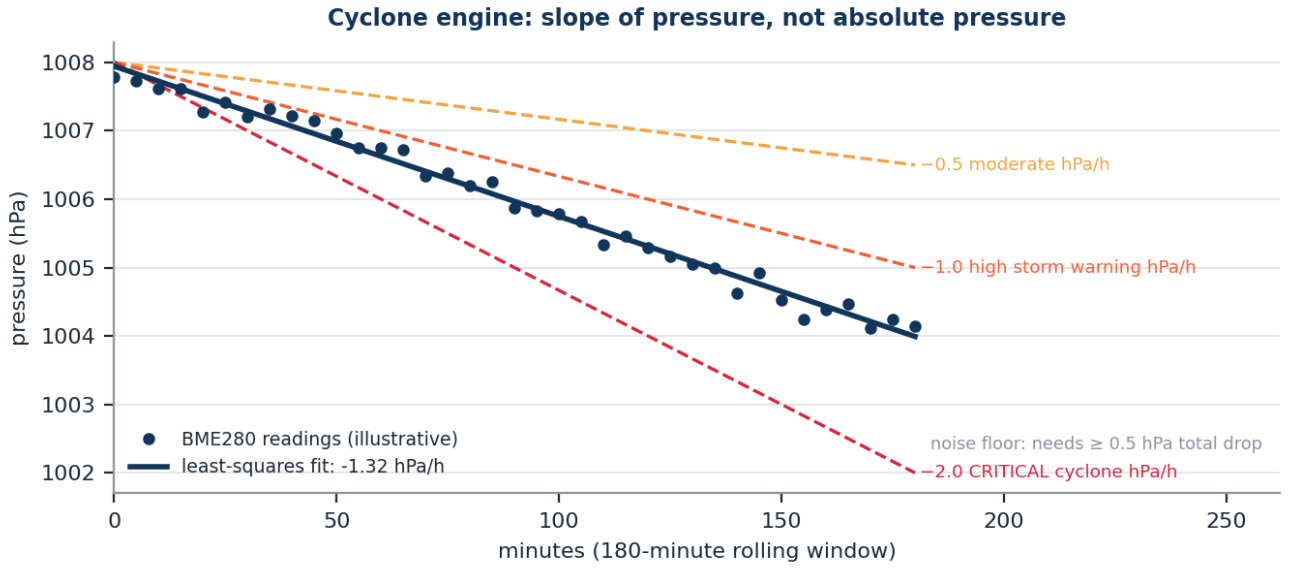



*Figure 14.1 — The fitted slope is compared with three reference rates (illustrative data).*

| Level | Fall rate | 3-hour drop | Meaning |
|---|---|---|---|
| Moderate low pressure | ≤ −0.5 hPa/h | 1.5 hPa | Weather changing |
| High storm warning | ≤ −1.0 hPa/h | 3.0 hPa | Standard maritime gale warning |
| **Critical cyclone** | **≤ −2.0 hPa/h** | 6.0 hPa | Rapidly deepening cyclone precursor |

A **noise floor** requires a minimum absolute drop of 0.5 hPa so sensor drift or an elevator ride cannot trigger an alarm.

## 14.3 Heat and smoke engines

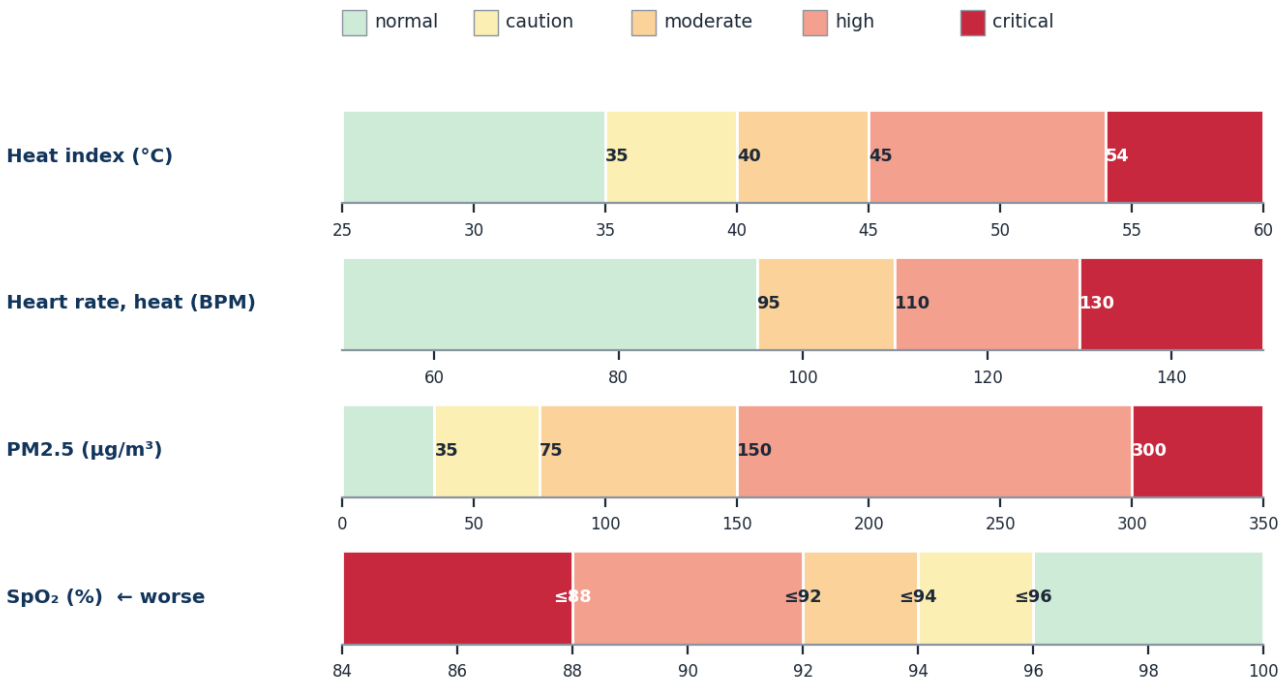



*Figure 14.2 — Threshold ladders. Any one rung can raise the level; the highest level reached wins.*

- **Heat strain:** NOAA/NWS Rothfusz Heat Index (clamped to 58 °C to avoid divergence) combined with cardiac response — Critical (heat stroke imminent) at HI ≥ 54 °C or HR ≥ 130 BPM or RMSSD ≤ 10 ms.
- **Toxic smoke:** PM2.5 fused with desaturation — Critical (severe respiratory distress) at PM2.5 ≥ 300 µg/m³ or $SpO_2$ ≤ 88 % or HR ≥ 120 BPM. The 35 µg/m³ caution level is the US EPA 24-hour standard (WHO 2021 guideline: 15 µg/m³).
- **Environmental AQI floor:** even if a fit rescuer has not desaturated yet, hazardous air sets a floor — EPA AQI 151–200 → MODERATE, 201–300 → HIGH, 301+ → CRITICAL.

| Level | Heat strain (any one) | Smoke (any one) |
|---|---|---|
| **Caution** | Heat Index ≥ 35 °C | PM2.5 ≥ 35 µg/m³ or $SpO_2$ ≤ 96 % |
| **Moderate** | HI ≥ 40 °C or HR ≥ 95 or RMSSD ≤ 35 ms | PM2.5 ≥ 75 or $SpO_2$ ≤ 94 % |
| **High** | HI ≥ 45 °C or HR ≥ 110 or RMSSD ≤ 20 ms | PM2.5 ≥ 150 or $SpO_2$ ≤ 92 % or HR ≥ 100 |
| **Critical** | HI ≥ 54 °C or HR ≥ 130 or RMSSD ≤ 10 ms | PM2.5 ≥ 300 or $SpO_2$ ≤ 88 % or HR ≥ 120 |

## 14.4 How the AI and the rules combine

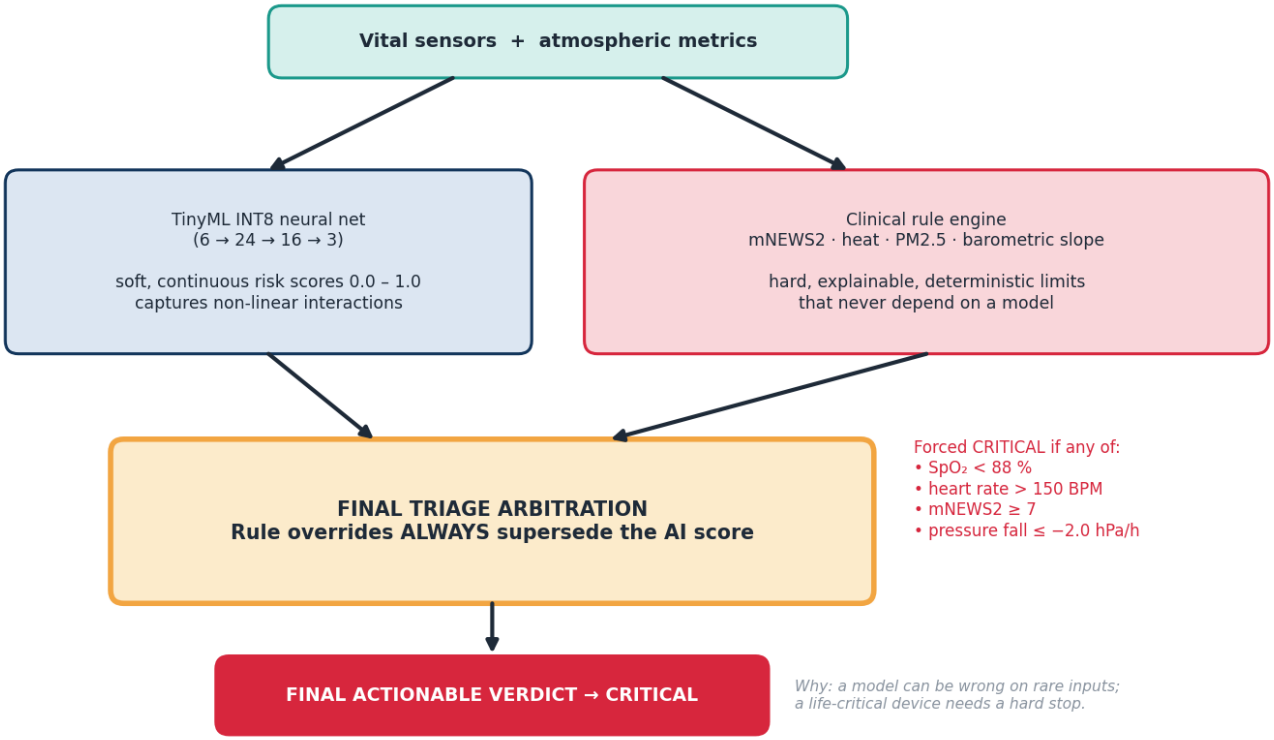



*Figure 14.3 — The hybrid safety envelope. Deterministic limits are an unbreakable hard stop over the neural network.*

Normally the INT8 network blends subtle multi-parameter interactions into smooth risk scores. But if $SpO_2$ falls below 88 %, heart rate exceeds 150 BPM, mNEWS2 reaches 7 or more, or pressure falls at −2.0 hPa/h or faster, the system forces CRITICAL, **bypassing the neural prediction**. Statistical edge cases must never be able to silence a genuine emergency.

**IN ONE LINE**

mNEWS2, barometric slope regression and environmental thresholds form an ironclad safety envelope that can override the AI during emergencies.

**JUDGE MIGHT ASK**

**Q.** *Why not let the network decide everything?*

**A.** Networks are statistical approximations that can behave unpredictably on out-of-distribution inputs. A life-critical device needs deterministic hard stops.

**Q.** *Why track fall rate and not absolute pressure?*

**A.** A mountain town has lower pressure than a coastal city. A drop of 2 hPa per hour signals a storm at any altitude.

**Q.** *Limits of mNEWS2 here?*

**A.** Standard NEWS2 uses AVPU consciousness and oxygen use, which need a human. VALOR scores the four sensor-measurable vitals and treats consciousness as alert unless a fall is confirmed.

PART III · SOFTWARE AND INTELLIGENCE

# 15 Sensors in Simple Terms

**AFTER THIS CHAPTER YOU CAN…**

- ✓ say what each sensor measures and on which bus
- ✓ explain how ratio-of-ratios gives $SpO_2$
- ✓ explain the two-source skin-temperature story and why NEWS2 temperature is withheld

| Sensor | Measures | Interface | Used for |
|---|---|---|---|
| **MAX30102** | IR (880 nm) and red (660 nm) light absorption | I²C, 100 Hz | Scaled to 8 bits → FPGA; heart rate; $SpO_2$ |
| **BME280** | Temperature, humidity, pressure | I²C, 1 Hz | Heat Index; 180-minute pressure regression |
| **PMS5003** | Laser scattering, 0.3–10 µm | UART1, 9600 baud, 1 Hz | PM1.0/2.5/10; 60 s rolling EPA/AirGradient correction |
| **MPU6050** | ±8 g acceleration, ±2000 °/s rotation | I²C, 40 Hz | Fall FSM; motion energy for PPG artefact rejection |
| **u-blox NEO-M8N / L89HS** | GPS / GLONASS signals | UART2, 9600 baud, 1 Hz | Latitude and longitude |
| **SSD1306** | 128 × 64 monochrome OLED | I²C, 1 Hz | Vitals, alerts, emergency card |

## 15.1 How a light sensor measures oxygen

Oxygenated and deoxygenated haemoglobin absorb red and infrared light differently. Each light signal has a steady (DC) part from tissue and a small pulsing (AC) part from arterial blood. The **ratio of the two ratios** is R, and $SpO_2$ follows from R.

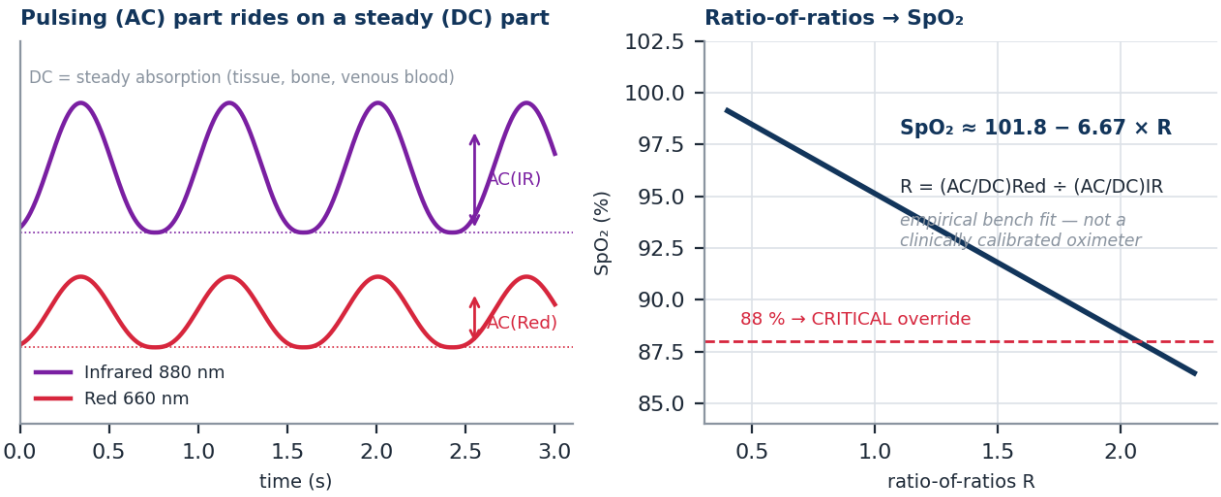



*Figure 15.1 — AC/DC components and the empirical mapping from R to $SpO_2$ used on the bench.*

**HONEST DISCLOSURE**

$SpO_2 \approx 101.8 - 6.67 \times R$ is an empirical bench fit. Clinical pulse-oximeter calibration needs controlled human hypoxia testing in a blood-gas laboratory, which we have not done.

## 15.2 Skin temperature: two sources, one honest label

- **Two sources.** The MAX30102's on-die sensor is always present. An optional TI TMP117 contact probe (±0.0078125 °C per LSB, device ID 0x0117, range −20…60 °C) sits on the shared I²C bus at 0x48–0x4B; the driver probes all four addresses at boot and takes over if one answers.
- **Every number carries its source.** `/status.json` publishes `skinsrc` as NONE, ESTIMATED or TMP117. The dashboard shows `Skin XX.XX °C (TMP117 probe)` for a measurement and `Skin ≈ XX.X °C` for an estimate. There is never a bare number.
- **The TMP117 story.** One breakout module never answered on 21 Oct 2026 even though the bus was healthy (3.18 V, four other devices clean) — a defective module, not a firmware fault. The driver was written so a missing probe degrades to the die estimate rather than blanking the field.
- **The die temperature is measured properly.** max30102_read_die_temp() uses bounded conversion polling and rejects an unconverted register (ti = 0 and low nibble 0) instead of accepting 0.0 °C. The LED self-heating offset was **measured**, not assumed: die 28.44 °C vs ambient 24.05 °C = **4.39 °C**. An earlier hand-guessed guard of ambient + 5.5 sat above the real peak (30.90 vs 30.07 °C), silently forcing the estimator to contribute exactly 0.00 °C — a real bug the measurement exposed.

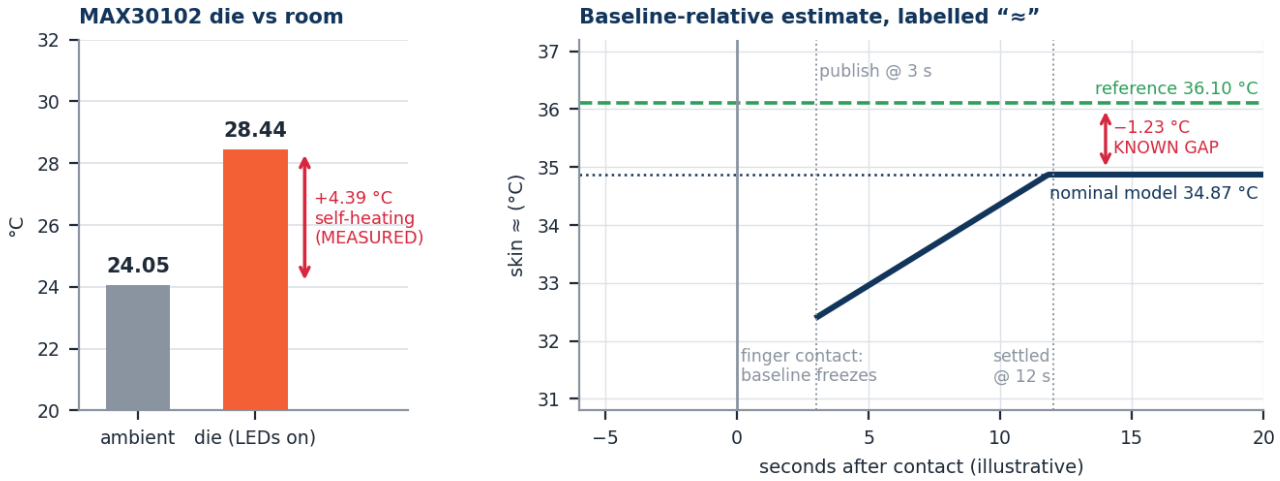



*Figure 15.2 — Left: the measured self-heating offset. Right: a schematic of the estimator's behaviour after finger contact (illustrative curve; endpoints from the bench).*

### The static-formula trap and the baseline-relative model

Putting the die temperature into a fixed linear equation fails because the LEDs (about 128,700 IR counts at the shipped setting) and the ESP32 PCB heat the idle die well above ambient. `skin_estimate.c` instead tracks the idle equilibrium before contact, freezes that baseline on contact, and uses the *relative step*:

$$T_{skin} = 34.10 + 0.55\cdot(T_{die_pred} - T_{die_baseline}) + 0.12\cdot(T_{amb} - 27.5)$$

with a 0.28 °C/s slew limit, a 5-sample median spike filter and monotonic-drift rejection. It publishes 3 s after contact and settles at 12 s; the earlier 4 s lead compensation and 34.50 °C base were removed for stability.

**HONEST DISCLOSURE**

**What it is worth.** Against a known reference the nominal model prints “[KNOWN GAP] model 34.87 °C vs reference 36.10 °C (error −1.23 °C)”. That is why the tile says *(uncalibrated)*.

**Why NEWS2 temperature stays at 0.** The NEWS2 temperature row is defined on core/tympanic values; a healthy fingertip reads 30–34 °C. Feeding it in would score +3 hypothermia on every well person and manufacture a critical alert. Only a probe at a core-equivalent site with a documented offset may feed the score.

**Calibration is blocked on data, not code.** tools/skin_calib_session.json holds two points both at 36.1 °C, which is degenerate. A real fit needs about 100 paired readings against a tympanic reference over 6+ sessions.

**IN ONE LINE**

We support a contact probe and the die sensor, we measured the LED offset instead of guessing, we label every number with its source, and we withhold the NEWS2 temperature row because a fingertip is the wrong site for a core-temperature score.

**JUDGE MIGHT ASK**

**Q.** *Do you have a skin-temperature sensor?*

**A.** Yes, two paths: the always-present MAX30102 die sensor (bench-measured offset 4.39 °C) and an optional TMP117 probe whose driver auto-detects at boot. The dashboard labels which is in use.

**Q.** *So what is your body-temperature reading?*

**A.** An uncalibrated peripheral skin estimate, labelled as such. We do not publish a core temperature: a fingertip is not a core site and the nominal model runs 1.23 °C low.

**Q.** *Why not use die temperature for fever detection?*

**A.** The die measures its own silicon, heated by the LEDs. Only the relative step from the pre-contact baseline means anything, and absolute thresholds would false-alarm.

**Q.** *How do you stop motion artefacts corrupting vitals?*

**A.** A signal-quality index (skewness term, Elgendi 2016, plus perfusion and IBI terms) and MPU6050 motion energy. If motion or poor perfusion is seen the device enters ACQUIRING and withholds HRV and clinical scores.

PART III · SOFTWARE AND INTELLIGENCE

# 16 Location and Connectivity

**AFTER THIS CHAPTER YOU CAN…**

- ✓ explain why GPS fails indoors and what VALOR does instead
- ✓ describe the three-tier location fallback

VALOR has a complete GNSS driver and NMEA-0183 parser (`gnss_nmea.c`, `gps_l89hs.c`; 12/12 unit tests pass) wired to a u-blox NEO-M8N / Quectel L89HS over UART2. But GPS signals are 1.5 GHz and very weak: they do not penetrate concrete, tunnels, basements or storm debris, and continuous polling costs 15–25 mA, which a wearable battery cannot afford.

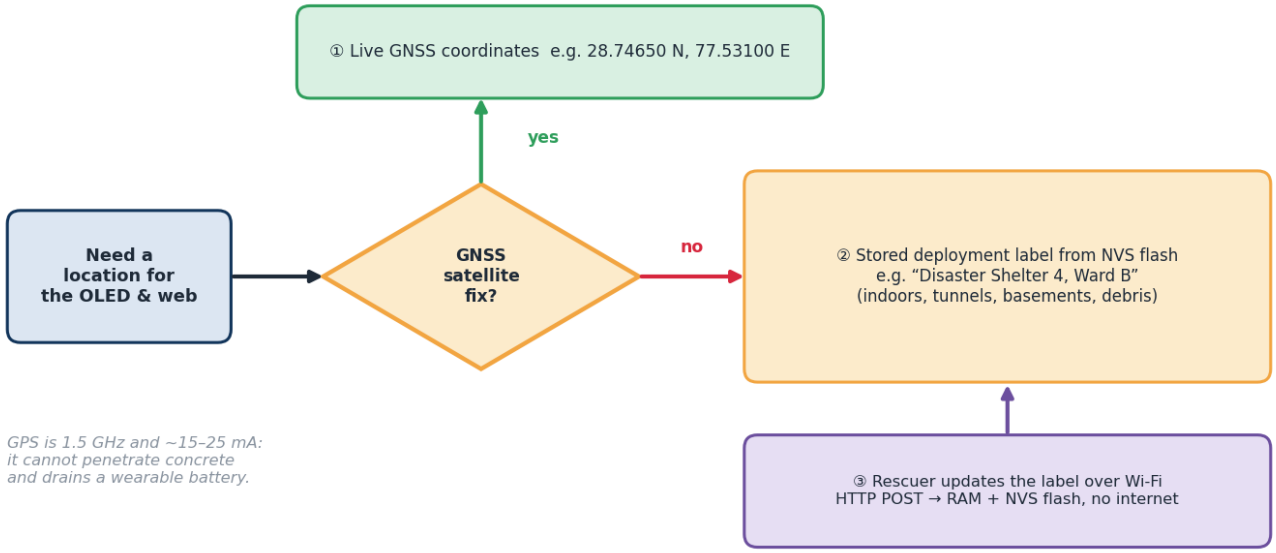



*Figure 16.1 — Three-tier location: live fix, stored label, or a label typed in by a rescuer.*

- ① **Live satellite fix** when outdoors, e.g. 28.74650 N, 77.53100 E.
- ② **Stored deployment label** in NVS flash (`location.c`) when there is no fix, e.g. “Disaster Shelter 4, Ward B”.
- ③ **Rescuer update**: a rescuer joins the SoftAP and types a new label; the ESP32 accepts the HTTP POST, updates RAM and writes NVS flash.

**Why this also helps the demo.** Inside an exhibition hall the receiver reports NO FIX. Rather than hang, VALOR shows its stored label, which demonstrates fault tolerance. And in a multi-storey shelter a label like “Field Hospital Tent 3, Bed 12” is more useful to a medic than a bare latitude and longitude.

**IN ONE LINE**

VALOR has a verified GNSS receiver but falls back to flash-stored or phone-entered locations indoors, where satellites cannot reach.

**JUDGE MIGHT ASK**

**Q.** *Why no GPS lock indoors?*

**A.** 1.5 GHz signals cannot penetrate concrete or debris and continuous polling draws about 20 mA. We use GNSS outdoors and a stored or typed label indoors.

**Q.** *How does a rescuer change the location without a network?*

**A.** Join the open VALOR-0081 Wi-Fi, type the label in the dashboard, submit. The ESP32 stores it in NVS flash in seconds.

PART IV · PROVE AND PRESENT

# 17 Verification and Testing

**AFTER THIS CHAPTER YOU CAN…**

- ✓ point to the evidence behind each technical claim
- ✓ state the honest status of each test

Every claim in VALOR is backed by an automated test, a bench measurement or a tool log. The last column is the one that matters in front of judges: it says honestly how far each result goes.

| Area | Method / tool | Evidence | Honest status |
|---|---|---|---|
| Zynq baseline | Vivado ML 2022.2 out-of-context synthesis | 185 LUT, 16 LUTRAM, 266 FF, 0 DSP, 0 BRAM; WNS +5.603 ns; Fmax 69.45 MHz | Verified golden reference |
| ForgeFPGA timing | Renesas fitter, SDC post-route | WNS +7.316 ns, TNS 0, 0 failing endpoints, Fmax 78.85 MHz | Timing closed at 50 MHz |
| ForgeFPGA resources | Fitter report resource_utilization_spi_link.log | 515/1120 LUT5 (45.98 %), 303 FF (16.33 %), 105/140 CLB (75 %), 0 BRAM, 0 PLL, 5 GPIO | Verified (bitstream 6930ba88…) |
| Verilog testbenches | Icarus Verilog (iverilog + vvp) | tb_forgefpga_system.v 13/13; tb_ppg_system.v 6/6 | 100 % passing |
| MIMIC-III benchmark | Host harness accuracy_evaluator.c | 16,387 records; 94.11 % accuracy, 98.19 % specificity, 58.18 % sensitivity; 99.95 % INT8 agreement | Validated; synthetic-RMSSD caveat |
| INT8 quantisation | GCC 15.2 host harness | MAE 0.00790, max error 0.09136, 97.32 % tier agreement over 7,000 samples | Fully verified |
| Clinical rules | test_disaster_risk_engine.c | 34 boundary-sweep tests ($SpO_2$, Heat Index, cyclone regression) | 100 % passing |
| Fall detection | Host unit tests + bench drop | 16 FSM tests; 1/1 physical breadboard drop on 2026-10-19 | Verified on hardware (single trial) |
| GNSS parser | test_gps_l89hs.c | 12/12 (checksums, coordinate signs, corrupted-packet rejection) | Verified in code and UAT |
| $I^2C$ bus | Oscilloscope + continuity report | 4 of 5 devices answer cleanly at 100 kHz; steady 3.18 V rail | Bus healthy; one breakout defective |

**IN ONE LINE**

Claims rest on test suites, Vivado and ForgeFPGA timing logs and 16,387 MIMIC-III records, with the bench gaps openly recorded.

**JUDGE MIGHT ASK**

**Q.** *How do you know INT8 does not drift from float?*

**A.** We ran both side by side on 16,387 records: 16,379 decisions identical (99.95 %).

**Q.** *Can you prove the Verilog passes right now?*

**A.** Yes. Run iverilog and vvp in PowerShell to execute all 13 self-checking assertions in tb_forgefpga_system.v live.

PART IV · PROVE AND PRESENT

# 18 Demo Walkthrough

**AFTER THIS CHAPTER YOU CAN…**

- ✓ run the five-act demo in about seven minutes
- ✓ know what each act proves and what to say
- ✓ recover when something fails on stage

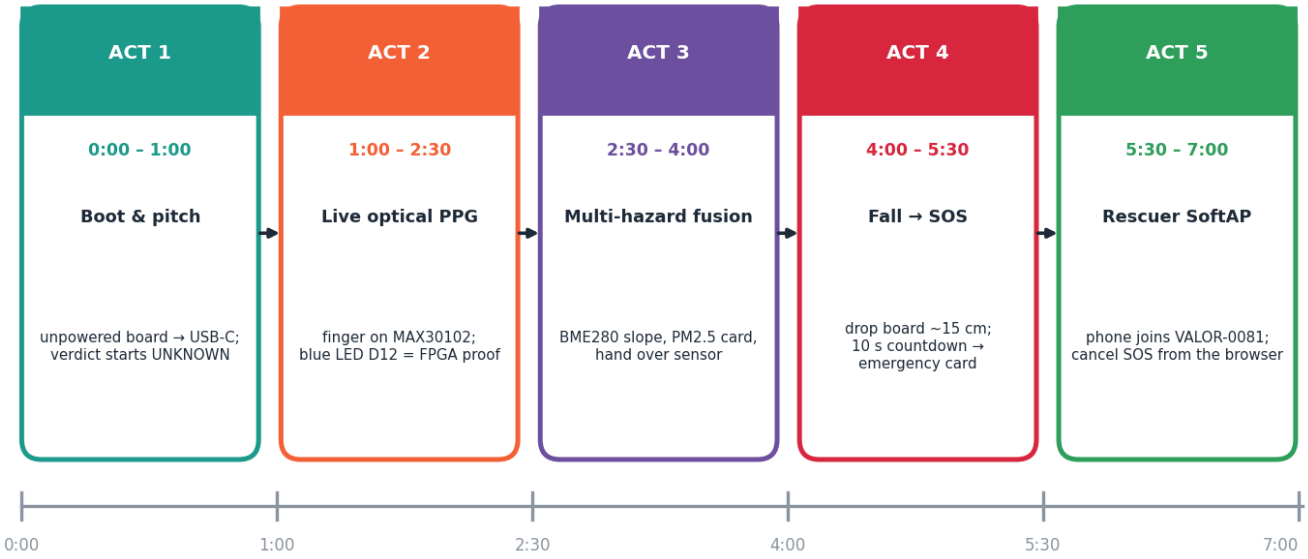



*Figure 18.1 — The five acts of the live demonstration (from DEMO_RUNBOOK.md).*

## Act 1 · Offline architecture and boot (0:00 – 1:00)

- Hold up the unpowered ShrikeFi board; point to the ForgeFPGA and the ESP32-S3. Plug in USB-C: the OLED initialises.
- Point out that before a finger is placed the verdict is **UNKNOWN (“Patient not assessed”)**. VALOR deliberately refuses to grade someone it has not measured.
- *Say:* “In a disaster the cloud does not exist. VALOR is a dual-brain wearable: a sub-dollar ForgeFPGA does optical conditioning with 20 ns timing; a dual-core ESP32-S3 does triage and an INT8 neural network, 100 % offline.”

## Act 2 · Hardware jitter elimination and live PPG (1:00 – 2:30)

- Finger firmly on the MAX30102. The blue LED D12 flashes with each pulse; the OLED shows HR (e.g. 74 BPM) and $SpO_2$ (e.g. 98 %). Open launch_dashboard.bat for the 60 FPS Web-Serial oscilloscope.
- *Say:* “That LED is not driven by software. It comes straight from our FPGA's peak detector. Operating systems add about 5 ms of jitter that destroys HRV; our FPGA timestamps peaks against a 50 MHz clock to 20 ns.”

## Act 3 · Multi-hazard fusion and storm tracking (2:30 – 4:00)

- Point at the BME280 and the pressure-trend gauge; cup a hand over the sensor to show temperature and humidity responding.
- *Say:* “VALOR tracks pressure over a 180-minute regression window; if it falls faster than 1 hPa per hour it raises cyclone risk and tells the user to seek shelter.”
- Show the PM2.5 card with its calibrated mass and confidence flag (PM25_CONF_OK), not raw scattering.

## Act 4 · Fall detection and SOS latching (4:00 – 5:30)

- Drop the board about 15 cm onto a soft notebook (or a rapid drop, sharp stop, then 1.5 s of stillness). The OLED shows *** **FALL DETECTED** *** and a 10-second ARMED countdown.
- *Say:* "The fall FSM just registered free-fall, impact and immobility. It is in a 10-second grace window in case someone just dropped a bag." Let the timer expire.

```
* MEDICAL EMERGENCY *
FALL DETECT T+00:15
HR 76 bpm  SpO2 98%
RR --/min  SQI 0.88
CONDITION: CRITICAL
LOC: KIET COLLEGE...
CALL 108 DO NOT MOVE
```

*The full-screen Emergency Assistance Card. The siren sounds at the same time.*

## Act 5 · Zero-install rescuer dashboard (5:30 – 7:00)

- On a phone, join `VALOR-0081`, open `http://192.168.4.1`. The dark-mode page loads from the ESP32's flash and shows **MEDICAL EMERGENCY: SOS ACTIVE (fall)**. Tap **CANCEL SOS**; the OLED returns to normal.
- *Say:* "My phone downloaded no app and there is no router in this room. The wearable is its own web server."

## If it fails on stage

| Symptom | Likely cause and fix |
|---|---|
| **Optical sensor shows NO_FINGER permanently** | Fluorescent room light saturates the photodiode — cup your hand over the sensor. |
| **OLED stays black** | Check I²C jumpers and the 3.3 V supply; power-cycle the USB connection. |
| **Phone will not connect** | Turn off mobile data so the phone does not drop a Wi-Fi network that has no internet. |
| **Hardware fails completely** | Switch to the simulation fallback: open valor_dashboard.html in Chrome, choose a simulated profile (Heat Wave & Dehydration, Severe Smog Crisis, Flash Flood), and run iverilog to show the logic live. |

**IN ONE LINE**

Five acts: offline boot, FPGA pulse detection, multi-hazard tracking, simulated fall with SOS escalation, and a zero-install phone dashboard.

**JUDGE MIGHT ASK**

**Q.** *How do you prove the FPGA detects the heartbeats, not the ESP32?*

**A.** The blue LED is wired to the FPGA output pad. Holding the FPGA in reset stops the flash immediately while the ESP32 keeps logging.

**Q.** *What if a wire comes loose?*

**A.** The dashboard has a simulation mode with verified clinical waveforms so the full analytical pipeline can still be shown.

**Q.** *How long until a stable vital lock?*

**A.** Optical acquisition begins within 10 ms of contact and the filter converges in 80 ms. Heart rate settles after about 2 pulses. HRV stabilisation is covered in Appendix C (the source gives two different figures).

PART IV · PROVE AND PRESENT

# 19 Judge Q&A Playbook — 30 Likely Questions

**AFTER THIS CHAPTER YOU CAN…**

- ✓ give a 20-second answer to each question
- ✓ add the follow-up detail when pressed
- ✓ avoid the phrases that lose credibility

Format: **20-second answer** first; *follow-up* if they dig; and the **don't say** line — the tempting wrong answer.

## Group 1 — Concept and architecture

**1. Why an FPGA if the ESP32-S3 runs at 240 MHz?**
**Answer.** FreeRTOS scheduling adds about 5 ms of jitter. RMSSD needs microsecond-level timing. The FPGA detects peaks in pure hardware at 50 MHz with 20 ns timestamps.
*Follow-up.* The tick quantum is 5 ms (CONFIG_FREERTOS_HZ = 200). Normal RMSSD is 10–50 ms, so 5 ms is a 10–50 % error.
**Don't say:** “The ESP32 is too slow.” The issue is scheduling jitter, not clock speed.

**2. How is VALOR different from a smartwatch?**
**Answer.** Watches need the cloud, lack smoke and barometric-storm sensing, suffer RTOS jitter and cannot host an open local triage page.
*Follow-up.* VALOR fuses vitals with heat, humidity, PM2.5 and pressure drops and runs the triage on-device.
**Don't say:** “Smartwatches are bad.” Frame it as different operating context.

**3. How does it operate with zero internet?**
**Answer.** Algorithms, INT8 weights and the HTML dashboard are all in internal flash; the ESP32 hosts its own Wi-Fi access point.
*Follow-up.* A phone at 192.168.4.1 talks over local radio to an embedded HTTP server.
**Don't say:** “It uses satellite internet.” It has none.

## Group 2 — FPGA, Verilog and RTL

**4. Moore or Mealy?**
**Answer.** Registered Moore: beat_detected is set inside a clocked block purely from being in STATE_PEAK_FOUND.
*Follow-up.* It never depends combinationally on sample_in or dyn_threshold → a glitch-free one-cycle pulse.
**Don't say:** “Mealy because inputs cause transitions.” Moore/Mealy is about how outputs are produced.

**5. Why binary, not one-hot, encoding?**
**Answer.** Binary uses 2 flip-flops for 4 states; one-hot needs 4. On a 1,120-LUT FPGA flip-flops matter.
*Follow-up.* One-hot simplifies decode in big FPGAs with spare flip-flops.
**Don't say:** “Binary is faster.” One-hot is usually faster to decode.

**6. How do you handle clock-domain crossing?**
**Answer.** SPI pins are never clock nets; the FPGA oversamples sck, ss_n and mosi at 50 MHz through 2-flip-flop synchronisers.
*Follow-up.* Edges are detected synchronously (sck_r, sck_f) and the shifters act on those one-cycle pulses.
**Don't say:** “We wired SPI SCK to a clock pin.” It stops between bytes.

**7. What is WNS and what did you get?**
**Answer.** Slack is the margin between required and actual arrival time; WNS is the slowest path's slack. +7.316 ns on ForgeFPGA, +5.603 ns on Zynq.
*Follow-up.* WNS positive and TNS = 0 → runs reliably to 78.85 MHz against a 50 MHz target.
**Don't say:** “A small negative slack is fine.” Any negative slack violates setup.

**8. Why is BRAM zero and what is the real bottleneck?**
**Answer.** The filter and FSM fit in distributed flip-flops. The binding limit is CLB occupancy: 105/140 (75 %) while LUT use is only 515/1120 (45.98 %).
*Follow-up.* 342 → 515 LUTs came from the 32-bit IBI counter, burst serialiser and dummy-MOSI suppression; 8 BRAMs remain free.
**Don't say:** “The FPGA has no BRAM / is out of logic.”

**9. How does ÷8 cost no logic?**
**Answer.** Dividing by 8 is a 3-bit right shift — output wires are connected to higher bit positions.
*Follow-up.* Bit 3 of next_sum goes to bit 0 of data_out, bit 4 to bit 1, and so on.
**Don't say:** “The compiler optimises an integer divide.” There are no instructions in hardware.

## Group 3 — SPI link and interconnect

**10. What is the SPI frame?**
**Answer.** Each sample is one full-duplex byte: raw sample on MOSI, {beat_latched, filt[6:0]} back on MISO.
*Follow-up.* If bit 7 is set, a 5-byte burst returns the 32-bit IBI cycle count.
**Don't say:** “It sends a command packet with headers.” The transfer is the sample.

**11. Why was the 4-bit nibble protocol retired?**
**Answer.** It would use 6–8 of 19 GPIOs, need bit-banging and ignore the MCU's hardware SPI.
*Follow-up.* Mode 0 SPI needs 4 wires and an 8 µs transfer.
**Don't say:** “We still use the nibble bus.” It was never built.

**12. What if SPI exceeds 15 MHz?**
**Answer.** It fails: the FPGA oversamples SCK with 50 MHz, so SCK must stay well below 25 MHz (≤ ~10 MHz in practice).
*Follow-up.* Near 15 MHz the SCK period approaches the sampling period and edges are missed.
**Don't say:** “The wires melt.” It is a sampling limit.

## Group 4 — Sensors and interfaces

**13. Status of the TMP117 probe?**
**Answer.** The driver ships (16/16 host tests) and probes 0x48–0x4B at boot. One breakout never answered on a healthy bus — a defective module. If a probe answers it takes over; otherwise the die estimate stands in.
*Follow-up.* Either way the source is labelled (skinsrc: NONE / ESTIMATED / TMP117).
**Don't say:** “We gave up on it.” The driver was written because it failed.

**14. Why not read body temperature from the die sensor?**
**Answer.** It measures silicon heated by the LEDs; only the relative step from a pre-contact baseline is meaningful. We measured the offset at 4.39 °C.
*Follow-up.* We refuse to convert it to a NEWS2 score: a fingertip is not a core site. The model also runs 1.23 °C low.
**Don't say:** “The MAX30102 has no temperature sensor.” It does.

**15. How does the cyclone engine avoid false alarms?**
**Answer.** A least-squares slope over 180 minutes; a fall faster than 1 hPa/h with at least 0.5 hPa total drop triggers an alert.
*Follow-up.* Rate of fall removes altitude effects; the long window filters noise.
**Don't say:** “It alerts below 1000 hPa.” Pressure depends on elevation.

## Group 5 — TinyML, AI and clinical logic

**16. Is the neural network diagnosing disease?**
**Answer.** No. It computes multi-hazard physiological risk triage; absolute safety is governed by mNEWS2 rules.
*Follow-up.* If vitals breach critical limits, deterministic rules override the model.
**Don't say:** “It diagnoses sepsis and cardiac arrest.”

**17. What does 94.11 % on MIMIC-III mean?**
**Answer.** It shows the embedded pipeline and INT8 model faithfully reproduce our triage rules on 16,387 real ICU records, with 99.95 % FP32 equivalence. Always pair it with 58.18 % sensitivity / 98.19 % specificity.
*Follow-up.* The RMSSD column is synthetic because MIMIC-III v1.4 has no high-frequency waveforms.
**Don't say:** “It is 94.11 % accurate at replacing doctors.”

**18. Why remove the 3→8→4→1 PM2.5 network?**
**Answer.** It was unvalidated, collapsed small readings to zero in humid air and could hide toxic smoke. It was replaced with peer-reviewed EPA/AirGradient equations.
*Follow-up.* The correction uses a 60 s rolling mean and HUMID / DUST / SETTLING flags.
**Don't say:** "It was too big for flash." It was tiny — it was unsafe.

**19. What is INT8 quantisation and why use it?**
**Answer.** Weights stored as signed 8-bit integers cut flash by 75 % to 619 bytes with MAE 0.00790 (max 0.0914) versus FP32.
*Follow-up.* At inference the FPU dequantises on the fly; accuracy 88.39 % vs 88.47 %.
**Don't say:** "It makes the model more accurate." It trades 0.08 % for 75 % memory.

## Group 6 — Firmware and concurrency

**20. How do you prevent races between cores?**
**Answer.** Shared structures (IBI buffer, triage status) are guarded by the FreeRTOS mutex s_data_mutex.
*Follow-up.* Core 0 holds it for microseconds; Core 1 takes local snapshots.
**Don't say:** "No locks needed because it is dual-core." Cores share memory.

**21. What if a task runs out of stack?**
**Answer.** A stack-overflow abort. We profiled usage and allocated 10 KB (monitor) and 7 KB (accelerator).
*Follow-up.* CONFIG_FREERTOS_CHECK_STACK_OVERFLOW makes any overflow an identifiable abort.
**Don't say:** "FreeRTOS stacks grow automatically."

## Group 7 — Cost, power and viability

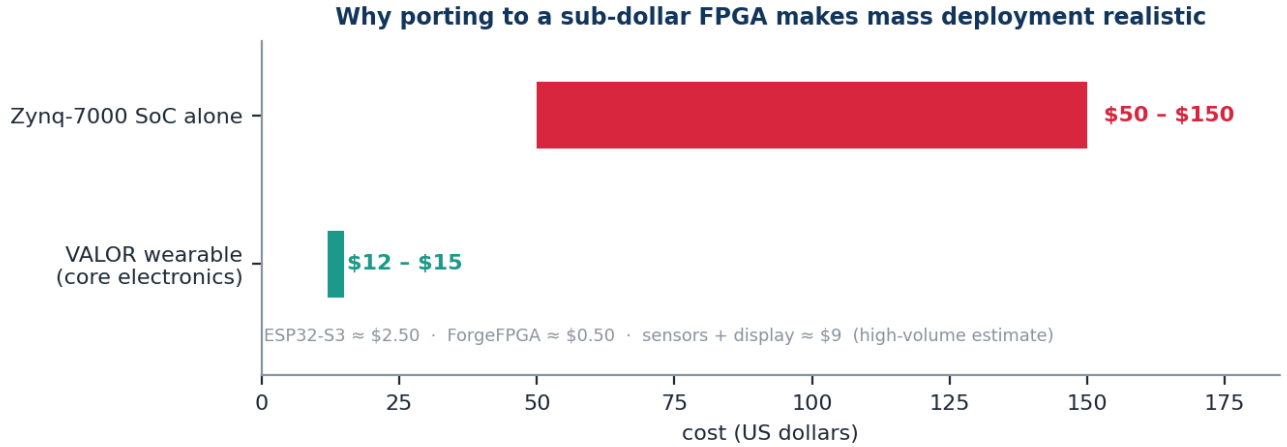



*Figure 19.1 — Core electronics cost versus a Zynq-7000 SoC alone.*

**22. What is the BOM cost?**
**Answer.** About $12–15 in volume: ESP32-S3 $2.50, ForgeFPGA $0.50, sensors and display about $9.
*Follow-up.* A Zynq-7000 alone is $50–150, so the port makes mass deployment viable.
**Don't say:** "It costs $2."

**23. Power and battery life?**
**Answer.** About 80–120 mA active; a 1,200 mAh LiPo gives 10–14 hours.
*Follow-up.* The FPGA draws under 5 mW; the big loads are Wi-Fi (~80 mA transmitting) and the PMS5003 fan (~100 mA). See Appendix C on duty cycling.
**Don't say:** "A month on a coin cell."

## Group 8 — Limitations and edge cases

**24. Why withhold HRV for the first 3 minutes?**
**Answer.** RMSSD needs a stable window of about 240 valid beats; an estimate from 10 s of data is meaningless. The display shows "settling n/240".
*Follow-up.* The rule engine and mNEWS2 run normally throughout because neither uses RMSSD.
**Don't say:** "The FPGA is slow at HRV." It gives IBIs immediately.

### 25. What about arrhythmia?
**Answer.** The 250 ms refractory handles rates to 240 BPM, but erratic IBIs lower the SQI and flag the reading.
*Follow-up.* VALOR is a screening device, not a 12-lead ECG; the OLED advises clinical verification.
**Don't say:** “Our AI detects all arrhythmias.”

### 26. Dark skin or calloused hands?
**Answer.** Darker pigment absorbs more light. The driver sets a calibrated static LED current of 11.0 mA (register 0x37, ceiling 18.0 mA 0x5A) to keep perfusion stable without excess heating.
*Follow-up.* If amplitude is still too low, SQI falls below 0.70 and the reading is withheld.
**Don't say:** “Skin colour does not affect optical sensors.”

### 27. Why no battery in the demo unit?
**Answer.** USB-C gives steady rails for serial telemetry and probing; it also avoids LiPo hazards on the bench.
*Follow-up.* The schematic has 3.3 V LDO pads and a single-cell LiPo management header.
**Don't say:** “We forgot the battery.”

### 28. Fall versus dropped phone?
**Answer.** A 3-stage sequence: free-fall (< 0.6 g), impact (> 2.5 g) and ≥ 1.5 s of stillness (1 g ± 0.3 g), then a 10 s cancel window.
*Follow-up.* A dropped object lacks the rotation dynamics and the vital coupling of a worn device.
**Don't say:** “It detects falls perfectly.”

### 29. Can the bitstream be updated in the field?
**Answer.** Yes. The ESP32 loads the SRAM-based ForgeFPGA over SPI at every boot, so an OTA firmware update updates the bitstream.
*Follow-up.* The bitstream is a 46,408-byte array in flash; no JTAG needed.
**Don't say:** “The FPGA is permanently burned.”

### 30. What would an auditor attack first?
**Answer.** The lack of a core-equivalent temperature probe, the synthetic MIMIC RMSSD column, and motion artefacts on fingertip PPG.
*Follow-up.* All three are in our risk register and mitigated by SQI gating, labelled sensor status and bounded claims.
**Don't say:** “Nothing; it is unbreakable.”

PART IV · PROVE AND PRESENT

# 20 Honest Limitations

To survive a rigorous panel, state the project's boundaries before anyone asks. Never guess and never over-claim.




*Figure 20.1 — Define the claim narrowly and defend it fully.*

| # | Limitation | Detail |
|---|---|---|
| 1 | Not a certified medical device | An emergency screening and triage tool: no clinical trials, no FDA 510(k), no CDSCO certification. It gives early-warning alerts, not diagnoses. |
| 2 | No core-equivalent temperature probe | Only a fingertip skin estimate (die sensor with measured 4.39 °C offset, plus an optional TMP117). No tympanic/axillary probe and no gold-standard calibration; the model runs 1.23 °C low. NEWS2 temperature points stay 0 and cold stress is an exposure proxy capped at HIGH. |
| 3 | Synthetic RMSSD in MIMIC validation | HR and $SpO_2$ are real; the RMSSD column was generated because MIMIC-III v1.4 has no raw waveform tables. |
| 4 | Motion artefacts on fingertip PPG | SQI gates corrupted samples, but vigorous arm motion or running forces the device to withhold vitals. |
| 5 | No battery in the demo | USB-C power only. A production unit needs a custom 4-layer rigid-flex PCB with LiPo charge controller and PMIC. |
| 6 | $SpO_2$ calibration | $SpO_2 \approx 101.8 - 6.67 \times R$ is an empirical bench fit; clinical calibration needs controlled-hypoxia testing. |
| 7 | PM2.5 calibration is inherited | We apply the published EPA/AirGradient equations; with no reference beta-attenuation monitor we inherit their published performance. |

**IN ONE LINE**

VALOR is an engineering prototype for offline disaster triage, not an FDA-certified diagnostic device; its boundaries are openly acknowledged and defended.

**JUDGE MIGHT ASK**

**Q.** *Is VALOR a certified clinical monitor?*

**A.** No. It is an edge engineering proof of concept. Clinical approval would need ISO 13485 manufacturing, IEC 60601-1 electrical safety, multi-centre prospective human trials and biocompatible housing.

**Q.** *Why was the battery left out of the demo?*

**A.** USB gives steady rails for probing and avoids lithium fire hazards on the bench. A production unit would use a LiPo with a PMIC (see Appendix C for the battery-life figures).

**Q.** *Main limitation of fingertip PPG?*

**A.** Motion artefacts. We mitigate with MPU6050 motion gating and SQI, withholding vitals when motion is high.

PART IV · PROVE AND PRESENT

# 21 Glossary

| Term | Meaning |
|---|---|
| **ADC** | Analog-to-digital converter: turns a voltage into a number. |
| **ALU** | Arithmetic logic unit: the integer-math circuit inside a CPU. |
| **AXI4-Lite** | Simple memory-mapped bus from ARM's AMBA family, used on the Zynq. |
| **BME280** | Bosch sensor for temperature, humidity and barometric pressure. |
| **BPM** | Beats per minute. |
| **BRAM** | Block RAM: dedicated SRAM blocks inside an FPGA. |
| **CDC** | Clock-domain crossing: passing a signal between unrelated clocks. |
| **CLB** | Configurable logic block: a tile of LUTs and flip-flops (8 LUT5s on ForgeFPGA). |
| **CPHA / CPOL** | SPI phase and polarity: which edge samples, and the idle level of the clock. |
| **DSP slice** | Hard multiplier-accumulator block in an FPGA. |
| **FF** | Flip-flop: a 1-bit memory element updated on a clock edge. |
| **Fmax** | Highest clock frequency a design can run at without timing violations. |
| **ForgeFPGA** | Renesas family of sub-dollar, ultra-low-power FPGAs. |
| **FPU** | Floating-point unit. |
| **FreeRTOS** | Real-time operating-system kernel for microcontrollers. |
| **FSM** | Finite state machine: states, transitions and actions. |
| **GNSS** | Global navigation satellite systems (GPS, GLONASS, Galileo, BeiDou). |
| **HRV** | Heart-rate variability: beat-to-beat variation in timing. |
| **IBI** | Inter-beat interval: time between two systolic peaks. |
| **IMU** | Inertial measurement unit: accelerometer + gyroscope. |
| **INT8** | 8-bit signed integer (−128…127) used to store quantised weights. |
| **I²C** | Two-wire (SDA, SCL) shared bus with addressed devices. |
| **LUT** | Look-up table: a small memory implementing any logic function. |
| **MAX30102** | Maxim optical pulse-oximeter / heart-rate sensor. |
| **Metastability** | A flip-flop output hovering between 0 and 1 after a setup/hold violation. |
| **MIMIC-III** | Public ICU database from MIT and Beth Israel Deaconess. |
| **MLP** | Multilayer perceptron: a feed-forward neural network. |
| **mNEWS2** | Modified National Early Warning Score 2: a clinical illness-severity score. |
| **MOSI / MISO** | SPI data lines: master-out/slave-in and master-in/slave-out. |
| **MPU6050** | 6-axis IMU (3-axis accelerometer, 3-axis gyroscope). |
| **NMEA-0183** | ASCII sentence standard used by GPS receivers. |
| **NVS** | Non-volatile storage partition in ESP32 flash. |
| **OLED** | Display where each pixel emits its own light. |
| **PMS5003** | Plantower laser particulate sensor. |
| **PPG** | Photoplethysmography: optical measurement of blood-volume changes. |
| **ReLU** | Activation function f(x) = max(0, x). |
| **RMSSD** | Root mean square of successive differences: the main time-domain HRV measure. |
| **RTL** | Register-transfer level: hardware described as data moving between registers. |
| **SLG47910** | The ForgeFPGA device on ShrikeFi (1,120 LUT5s). |

| Term | Meaning |
|---|---|
| **SoftAP** | Mode in which the ESP32 acts as a Wi-Fi access point. |
| **SPI** | Four-wire synchronous bus: SCK, SS, MOSI, MISO. |
| **$SpO_2$** | Estimated percentage of oxygenated haemoglobin in arterial blood. |
| **SQI** | Signal quality index (0–1) rating a pulse waveform. |
| **TinyML** | Machine learning on low-power microcontrollers. |
| **TMP117** | TI high-precision (±0.1 °C) digital temperature sensor. |
| **UART** | Asynchronous serial link with matched baud rates and no clock wire. |
| **WNS** | Worst negative slack: the timing margin of the slowest path. |
| **Zynq-7000** | AMD Xilinx SoC pairing ARM Cortex-A9 cores with Artix-7 fabric. |

PART IV · PROVE AND PRESENT

# 22 Who Should Be Able to Explain What

Three spoken roles are defined in DEMO_RUNBOOK.md. Every teammate must own a technical area so the presentation and the defense are seamless.

| Teammate | Role | Master these chapters | Own these defense topics |
|---|---|---|---|
| **1** | Live hardware demo and sensor handling | 0, 1, 2, 3, 6, 15, 16, 18 | Demo walkthrough · sensor wiring and I²C behaviour · SoftAP and the zero-install dashboard · skin-temperature two-source story · fall trigger and cancel · indoor GPS fallback to NVS |
| **2** | RTL and digital-hardware architecture | 4, 5, 7, 8, 9, 10, 11, 17 | Zynq baseline vs ShrikeFi port · 4-wire SPI and the retired nibble bus · CDC and metastability · 8-tap filter and free ÷8 · peak-FSM cycle by cycle · 250 ms refractory · timing closure (WNS +7.316 ns), 0 DSP / 0 BRAM |
| **3** | Clinical triage and zero-cloud TinyML | 12, 13, 14, 17, 19, 20 | Core 0 vs Core 1 pinning · INT8 MLP (6→24→16→3, 619 B) · removal of the PM2.5 network · MIMIC-III 94.11 % always with 58.18 % / 98.19 % and the synthetic-HRV caveat · mNEWS2 · pressure-slope threshold · rules override AI |

PART IV · PROVE AND PRESENT

# 23 Open Questions and Reconciliations

In the spirit of engineering integrity, here is every place where the history of the project, the documents and the code differ — so you can defend each with confidence.

## 23.1 Documented in the original study text

- **Peak-detector names and drop rule.** An early deck (JUDGE_QA.md lines 96–100) used IDLE / ARMED / CREST_FALL / REFRACTORY with a fixed drop of 3. The synthesisable ppg_peak_detector.v uses ARMED (00), RISING (01), PEAK_FOUND (10), REFRACTORY (11) with a proportional drop (peak >> 2 for peak > 48, else 6). The Verilog is the only source of truth.
- **Resource evolution 342 → 363 → 362 → 535 → 515 LUT5s.** Early decks quote 342/1120 (30.54 %). The active post-route log quotes 515/1120 (45.98 %), 303 FFs, 105/140 CLBs. The rise is the Oct 18 IBI-counter / burst-serialiser upgrade (Chapter 11).
- **Nibble link vs real SPI.** docs/archive/MIGRATION_AGENT_TASKS.md proposes a 4-bit bus; it was retired before implementation. The real link is 4-wire 8-bit SPI Mode 0 (SPI2_HOST, 1 MHz runtime).
- **GNSS status.** Fully implemented and unit-tested (12/12); indoors it falls back to NVS flash text or an over-the-air rescuer update.
- **Skin temperature.** Two independent paths (die estimate and optional TMP117), an explicit "≈" label, skinsrc in /status.json, a known −1.23 °C gap, and NEWS2 temperature held at 0.
- **MIMIC-III waveform disclosure.** HR and $SpO_2$ are real chart data; the RMSSD column is synthetic.

## 23.2 Disclosures to volunteer before a judge finds them

| Topic | Reality |
|---|---|
| **Reset architecture** | por_cnt is synchronous to the 50 MHz clock but rst_n currently drives negedge rst_n asynchronously in submodules. A future revision will dual-flop-register it for purely synchronous resets. |
| **Weak-signal branch inactive** | crest_fall_min = (peak > 48) ? peak>>2 : 6, but arming needs sample ≥ 120, so the 25 % rule is always used and the 6-count floor is unreachable. |
| **Single-channel FPGA link** | Only IR goes to the FPGA; red is conditioned in software for $SpO_2$. |
| **First-beat suppression** | first_beat_seen suppresses the first crest after reset so it only seeds the interval counter, with no spurious beat flag or LED flash. |
| **MISO modulo-128** | Bit 7 is the beat flag, so bits [6:0] wrap at 128 on raw logs; diagnostic only. |
| **TinyML sparsity** | 391/619 zeros (63.17 %), 11 + 6 dead neurons: effective 6→13→10→3 at 88.39 % accuracy. |
| **Jitter source** | The 5.0 ms quantum comes from CONFIG_FREERTOS_HZ = 200 in firmware/shrikefi/sdkconfig.defaults. |
| **Wi-Fi core pinning** | CONFIG_ESP_WIFI_TASK_PINNED_TO_CORE_1 = y. |
| **MAX30102 interrupt** | Writing 0x40 to INT_ENABLE_1 (0x02) enables bit 6 PPG_RDY (per-sample), not bit 7 A_FULL. |
| **Zynq timing runs** | AGENTS.md golden baseline is out-of-context synthesis WNS +5.603 ns (Fmax 69.45 MHz). Full block-design runs in Vivado 2026.1 closed post-route at WNS +12.836 ns (1,806 endpoints) and +13.168 ns (1,826 endpoints). |

APPENDIX

# Appendix A · Quick-Reference Card

| Topic | Number to remember |
|---|---|
| **Link** | 4-wire SPI Mode 0, 1 MHz; 8 µs per byte every 10 ms (0.08 % duty); burst magic 0xC3 |
| **Timestamp** | 50 MHz clock → 20 ns resolution; ms = cycles ÷ 50,000 |
| **Filter** | 8 taps, 11-bit running sum, ÷8 = shift, group delay 3.5 samples = 35 ms, fill 80 ms |
| **Peak FSM** | 4 states, 2-bit binary, Moore; arm threshold 120; fall ≥ peak/4; refractory 12,500,000 cycles = 250 ms → ceiling 240 BPM |
| **ForgeFPGA** | 515/1120 LUT5 (45.98 %), 303 FF, 105/140 CLB (75 %), 0 DSP, 0 BRAM, 5/19 GPIO; WNS +7.316 ns; Fmax 78.85 MHz |
| **Zynq** | 185 LUT, 266 FF, 0 DSP, 0 BRAM; WNS +5.603 ns; Fmax 69.45 MHz |
| **Firmware** | Core 0: accelerator (pri 5, 7 KB, 100 Hz). Core 1: monitor (pri 2, 10 KB, 1 Hz), PMS/GPS/IMU (pri 3, 4 KB), Wi-Fi |
| **TinyML** | 6→24→16→3, 619 INT8 bytes (2,476 as float32), 391 zeros, effective 6→13→10→3; 88.39 % vs 88.47 %; ~20–50 µs |
| **MIMIC-III** | 16,387 records, 98 patients: 94.11 % acc, 98.19 % spec, 58.18 % sens, 99.95 % INT8 agreement |
| **SOS timers** | 15 s critical-vitals grace, 10 s fall grace, 5 s hold to cancel, 30 s recovery stand-down |
| **Fall FSM** | < 0.60 g → > 2.50 g and ≥ 80 °/s within 800 ms → 1 g ± 0.30 g for ≥ 1.5 s |
| **Cyclone** | 180-min regression: ≤ −0.5 / −1.0 / −2.0 hPa/h; noise floor 0.5 hPa |
| **mNEWS2** | 0–4 low, 5–6 medium, ≥ 7 high; any single 3 escalates |
| **Overrides → CRITICAL** | $SpO_2$ < 88 %, HR > 150, mNEWS2 ≥ 7, pressure ≤ −2.0 hPa/h |
| **Wi-Fi** | Open SoftAP VALOR-0081, http://192.168.4.1, /api/status polled each second |
| **Skin temp** | Die offset 4.39 °C (28.44 vs 24.05); model gap −1.23 °C; publishes 3 s, settles 12 s; NEWS2 temp = 0 |

APPENDIX

# Appendix B · Self-Test

Answer without looking back. Each is a question a judge could ask in a different form.

1. Why does the FPGA exist when the ESP32-S3 is faster? Give two numbers.
2. Draw SPI Mode 0 for one byte and mark sample and shift edges.
3. What does bit 7 of the MISO byte mean, and what follows when it is 1?
4. Why does a signal crossing into the FPGA clock go through two flip-flops?
5. Compute the bit width of the filter's running sum for 8-bit input and justify it.
6. Why is dividing by 8 free in hardware? What about dividing by 7?
7. List the four peak-detector states and the condition leaving each.
8. How many clock cycles is the 250 ms refractory window at 50 MHz?
9. What is slack? What do WNS = +7.316 ns and TNS = 0 tell you?
10. Why is CLB occupancy, not LUT count, the real constraint?
11. Why does a mutex protect the vitals structure?
12. How many parameters are in 6→24→16→3, and why does it fit in 619 bytes?
13. State the MIMIC-III result correctly, in one sentence, with the caveat.
14. Why was the PM2.5 neural calibrator removed?
15. Why is NEWS2 temperature held at zero?
16. What happens, step by step, between a fall and the siren?

APPENDIX

# Appendix C · Reconciliation Notes from the Editing Pass

While converting the study text into this guide I cross-checked its numbers. These items either disagree between sections or deserve a verification against the code and the newer companion documents (DEMO_SHEET.md is stated to be the most current). I have kept the original wording in the chapters and flagged the issue here rather than silently choosing one.

| # | Where | What I noticed | Suggested resolution |
|---|---|---|---|
| 1 | Ch. 18 Q&A vs Ch. 19 Q24 | HRV stability is quoted as “30 valid beats (~20–30 s)” in one place and as a settled window of about 240 beats (about 3 min, “settling n/240”) in another. | Check the firmware constant. Likely: first RMSSD at 30 beats vs publishing after 240 — if so, say both. |
| 2 | Ch. 19 Q23 vs Ch. 20 | Battery life: 1,200 mAh → 10–14 h at 80–120 mA, versus a production 500 mAh LiPo → 8–12 h “duty-cycled”. Also the quoted peaks (Wi-Fi ~80 mA + PMS5003 fan ~100 mA) already exceed the 80–120 mA average. | State the average current explicitly and say which loads are duty-cycled; at 500 mAh and 100 mA continuous the runtime is about 5 h. |
| 3 | Ch. 13 | CPU clock is 240 MHz in some places and 160 MHz for the 20–50 µs inference figure. | Quote latency with its configured clock (160 MHz). |
| 4 | Ch. 19 Q30 vs Ch. 15 / 20 | Q30 says “missing physical skin-temperature sensor”, but two skin sources exist; what is missing is a core-equivalent probe and a calibration. | Use the Ch. 20 wording. |
| 5 | Ch. 3 vs Ch. 9 | Firmware accepts IBIs for 30–250 BPM, while the refractory window caps hardware detection at 240 BPM. | Both true; the hardware limit binds first. |
| 6 | Ch. 10 | Fall detector called “three-stage” and “five-state”. | Three physical stages between two end states; both are correct. |

| # | Where | What I noticed | Suggested resolution |
|---|---|---|---|
| 7 | Ch. 5 Q&A, Ch. 19 Q12 | SPI limit stated as 30 MHz fail, > 15 MHz fail, ≤ 5–10 MHz works; the Nyquist bound is 25 MHz. | Canonical line: practical SCK ≤ ~10 MHz; 1 MHz in use. |
| 8 | Ch. 23 (original) | The 21 Sept build is “364” in a heading and “363” in the list and Ch. 11. | Used 363; confirm in the fitter log. |
| 9 | Ch. 20 Q&A | One question on certification was truncated in the supplied file; its wording here is inferred from the answer. | Check against the source. |
| 10 | Source formatting | Mermaid state diagrams and some LaTeX in the original ($SpO_2$, Fmax in Ch. 23) did not render or were corrupted. | Redrawn as figures and retyped in this guide. |